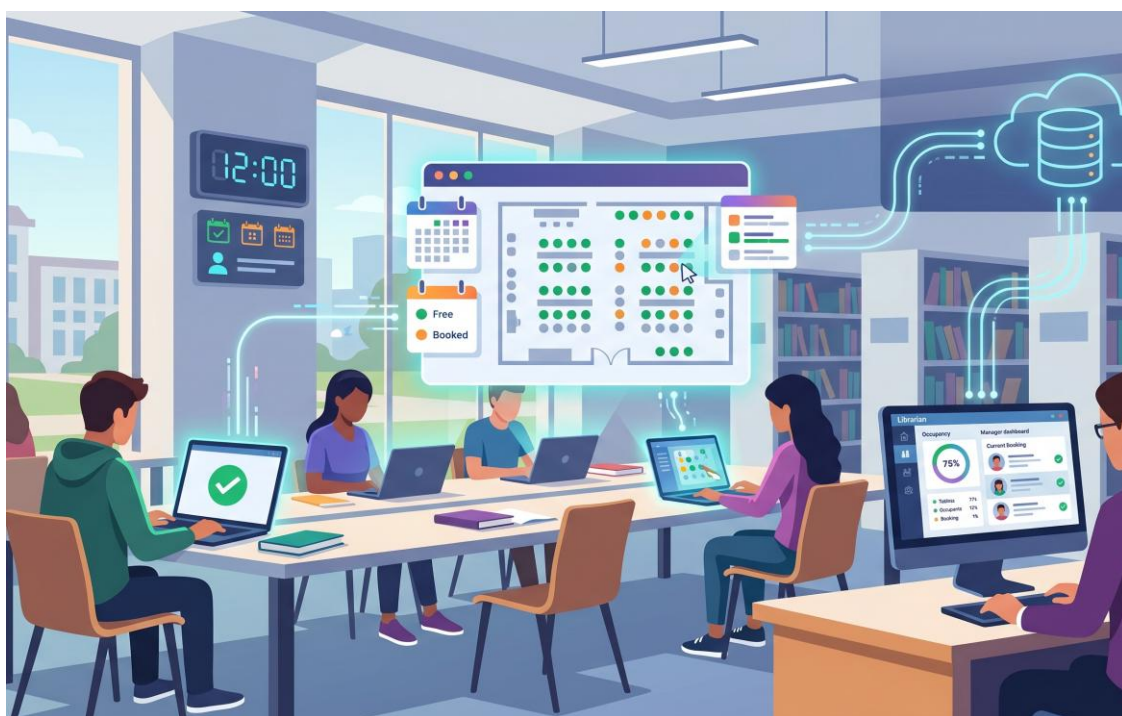


UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
CORSO DI LAUREA IN INGEGNERIA INFORMATICA
CORSO DI INGEGNERIA DEL SOFTWARE
PROF. A.R. FASOLINO - A.A. 2025-26...



**PROGETTO
DI INGEGNERIA DEL SOFTWARE**

**“SISTEMA SOFTWARE PER PRENOTAZIONI DI
POSTAZIONI IN BIBLIOTECA
UNIVERSITARIA”**

INDICE

1. SPECIFICHE INFORMALI	1
2. ANALISI E SPECIFICA DEI REQUISITI	3
2.1 ANALISI NOMI-VERBI	3
2.2 REVISIONE DEI REQUISITI	5
2.3 GLOSSARIO DEI TERMINI	6
2.4 CLASSIFICAZIONE DEI REQUISITI	7
2.4.1 Requisiti funzionali.....	7
2.4.2 Requisiti sui dati.....	8
2.4.3 Requisiti Non Funzionali di Qualità.....	9
2.4.4 Vincoli / Altri requisiti	10
2.5 MODELLAZIONE DEI CASI D'USO	11
2.5.1 Attori e casi d'uso.....	11
2.5.2 Diagramma dei casi d'uso.....	13
2.5.3 Scenari.....	13
2.6 DIAGRAMMA DELLE CLASSI.....	19
2.6.1 Responsabilità	19
2.7 DIAGRAMMI DI SEQUENZA	21
2.7.1 Registrazione	21
2.7.2 Autenticazione.....	21
2.7.3 Effettua Prenotazione	22
2.7.4 Crea Sala Studio.....	24
2.7.5 Elimina Sala Studio	25
2.7.6 Annulla Prenotazione.....	25
2.7.7 Effettua Check-in	26
2.7.8 VisualizzaProfiloPersonale.....	27
2.7.9 ConsultazioneSaleDisponibili	27
2.8 STATE CHART DIAGRAM	29
2.8.1 statoPrenotazione.....	29
2.8.2 statoPostazione.....	29
2.9 DIAGRAMMA DELLE CLASSI RAFFINATO	30
3. PIANO DI TEST FUNZIONALE	31
3.1 REGISTRAZIONE UTENTE.....	31
3.2 AUTENTICAZIONE UTENTE.....	33
3.3 EFFETTUA PRENOTAZIONE	34
3.4 CREAZIONE SALA STUDIO.....	39
3.5 ELIMINAZIONE SALA STUDIO	41
3.6 ANNULLAMENTO PRENOTAZIONE.....	42
3.7 CHECK-IN POSTAZIONE.....	42
3.8 VISUALIZZAPROFILOPERSONALE	42
3.9 CONSULTAZIONESALEDISPONIBILI	43
4. PROGETTAZIONE	44
4.1 WIREFRAMES.....	44
4.1.1 Homepage	44
4.1.2 Registrazione	44
4.1.3 Accesso.....	45

4.1.4 Dashboard Bibliotecario	45
4.1.5 Crea nuova Sala Studio	46
4.1.6 Aggiungi Area a Sala	46
4.1.7 Dashboard Studente	47
4.1.8 Profilo Personale	47
4.1.9 Effettua Prenotazione	47
4.2 DIAGRAMMA DELLE CLASSI	48
4.2.1 Traduzione classi ed associazioni	50
4.2.2 Pattern BCED	51
4.2.2.1 Package Boundary	51
4.2.2.2 Package Controller	51
4.2.2.3 Package Entity	52
4.2.2.4 Package Database	52
4.3 DIAGRAMMI DI SEQUENZA	53
4.3.1 Registrazione	54
4.3.2 Autenticazione	54
4.3.3 Effettua Prenotazione	54
4.3.4 Crea Sala Studio	54
4.3.5 Elimina Sala Studio	54
4.3.6 Annulla Prenotazione	54
4.3.7 Effettua Check-in	55
4.3.8 VisualizzaProfiloPersonale	55
4.3.9 ConsultazioneSaleDisponibili	55
5. IMPLEMENTAZIONE	56
5.1 PACKAGE DATABASE	56
5.2 PACKAGE ENTITY	56
5.3 PACKAGE CONTROLLER	56
5.4 PACKAGE BOUNDARY	56
5.5 PACKAGE DTO	56
5.6 DIAGRAMMA DI DEPLOYMENT	57
6. TESTING	58
6.1 TEST STRUTTURALE	58
6.1.1 Complessità ciclomatica	58
6.1.1.1 inserisciAutoModifiche – GestioneParcoAuto	58
6.2 JUNIT – TEST DI UNITÀ	61
6.3 TEST FUNZIONALE	62

1. Specifiche informali

Riportare la Traccia assegnata

Si desidera sviluppare un sistema software per la gestione della prenotazione di postazioni di studio all'interno di una biblioteca universitaria. Il sistema dovrà consentire agli studenti di prenotare una postazione in specifiche fasce orarie, visualizzare le proprie prenotazioni e accedere ai servizi della biblioteca in modo organizzato. Il sistema coinvolge due tipologie di utenti: studenti e bibliotecari, ciascuna con funzionalità e permessi specifici.

Il sistema consente la registrazione di utenti tramite autenticazione con credenziali personali. Al momento della registrazione, ciascun utente deve specificare il proprio ruolo, scegliendo tra studente o bibliotecario, oltre a fornire nome, cognome, email istituzionale e numero di matricola o codice identificativo interno.

Ogni bibliotecario ha la possibilità di creare e gestire una o più sale studio. Per ciascuna sala, mediante apposita interfaccia grafica, è possibile specificare un nome, una descrizione, il numero totale di postazioni disponibili e gli orari di apertura giornalieri. Ogni sala può essere suddivisa opzionalmente in aree differenti, ad esempio area silenziosa, area consultazione o area lavoro di gruppo. Il bibliotecario può inoltre visualizzare in ogni momento l'elenco delle prenotazioni effettuate per ciascuna sala e monitorarne lo stato.

Ogni studente autenticato dispone di una propria interfaccia nella quale può consultare le sale disponibili e verificare, per ciascun giorno, le fasce orarie prenotabili. Una prenotazione viene effettuata selezionando una sala, una data, una fascia oraria e, se prevista, una specifica area o una specifica postazione. Il sistema deve impedire che una stessa postazione venga assegnata contemporaneamente a più studenti nella medesima fascia oraria. Una volta completata la prenotazione, essa entra nello stato "attiva".

Lo studente può visualizzare all'interno del proprio profilo personale l'elenco delle prenotazioni effettuate, con le relative informazioni di sala, data, orario e stato. Deve inoltre essere possibile annullare una prenotazione entro un certo limite temporale precedente all'inizio della fascia prenotata. In caso di annullamento, la postazione torna automaticamente disponibile per altri utenti.

Nel giorno della prenotazione, lo studente può effettuare il check-in tramite una apposita interfaccia grafica, selezionando la prenotazione attiva e confermando la propria presenza. Per semplicità, si può supporre che il check-in debba essere effettuato entro un intervallo di tempo limitato rispetto all'orario di inizio della prenotazione. Se il check-in non viene effettuato entro tale intervallo, la prenotazione passa automaticamente nello stato "scaduta" e la postazione viene resa nuovamente disponibile.

Il bibliotecario, attraverso la propria interfaccia, può consultare in tempo reale la situazione delle sale, visualizzando il numero di postazioni occupate, il numero di postazioni libere e l'elenco delle prenotazioni attive per la giornata corrente. Deve inoltre essere possibile accedere a uno storico delle prenotazioni per ciascuna sala e per ciascuno studente.

Ogni studente ha accesso a un profilo personale che consente di consultare le prenotazioni future, quelle passate, quelle annullate ed eventualmente il numero totale di accessi effettuati alla biblioteca. I bibliotecari, attraverso un'interfaccia dedicata, possono monitorare l'andamento complessivo del servizio, visualizzando il tasso di occupazione delle sale, il numero di prenotazioni giornaliere e il numero di prenotazioni non confermate tramite check-in.

Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. È previsto un sistema di notifiche che avvisi gli studenti della conferma della prenotazione, di eventuali annullamenti e dell'approssimarsi dell'orario prenotato. Il sistema dovrà inoltre garantire la protezione dei dati personali e una corretta gestione dei permessi e delle visibilità tra studenti e bibliotecari.

2. Analisi e specifica dei requisiti

2.1 Analisi nomi-verbi

Il sistema consente la **registrazione** di **utenti** tramite **autenticazione** con credenziali personali. Al momento della **registrazione**, ciascun utente deve specificare il proprio **ruolo**, scegliendo tra **studente** o **bibliotecario**, oltre a fornire **nome**, **cognome**, **email istituzionale** e **numero di matricola** o **codice identificativo interno**.

Ogni **bibliotecario** ha la possibilità di **creare e gestire** una o più sale studio. Per ciascuna **sala**, mediante apposita interfaccia grafica, è possibile specificare un **nome**, una **descrizione**, il **numero totale di postazioni disponibili** e gli **orari di apertura** giornalieri. Ogni **sala** può essere suddivisa opzionalmente in aree differenti, ad esempio **area silenziosa**, **area consultazione** o **area lavoro di gruppo**. Il **bibliotecario** può inoltre **visualizzare in ogni momento l'elenco delle prenotazioni effettuate** per ciascuna **sala** e monitorarne lo stato.

Ogni **studente** autenticato dispone di una propria interfaccia nella quale può **consultare le sale disponibili** e verificare, per ciascun giorno, le **fasce orarie prenotabili**. Una **prenotazione** viene effettuata selezionando una **sala**, una **data**, una **fascia oraria** e, se prevista, una specifica **area** o una specifica **postazione**. Il sistema deve impedire che una stessa **postazione** venga assegnata contemporaneamente a più studenti nella medesima fascia oraria. Una volta completata la **prenotazione**, essa entra nello **stato** "attiva".

Lo **studente** può **visualizzare all'interno del proprio profilo personale l'elenco delle prenotazioni effettuate**, con le relative informazioni di **sala**, **data**, **orario** e **stato**. Deve inoltre essere possibile annullare una **prenotazione** entro un certo limite temporale precedente all'inizio della fascia prenotata. In caso di annullamento, la **postazione** torna automaticamente disponibile per altri **utenti**.

Nel giorno della **prenotazione**, lo **studente** può effettuare il check-in tramite una apposita interfaccia grafica, selezionando la **prenotazione** attiva e confermando la propria presenza. Per semplicità, si può supporre che il check-in debba essere effettuato entro un intervallo di **tempo** limitato rispetto all'orario di inizio della **prenotazione**. Se il check-in non viene effettuato entro tale intervallo, la **prenotazione** passa automaticamente nello **stato** "scaduta" e la **postazione** viene resa nuovamente disponibile.

Il **bibliotecario**, attraverso la propria interfaccia, può **consultare in tempo reale la situazione delle sale**, visualizzando il numero di postazioni occupate, il numero di postazioni libere e l'elenco delle prenotazioni attive per la giornata corrente. Deve inoltre essere possibile accedere a uno storico delle prenotazioni per ciascuna **sala** e per ciascuno **studente**.

Ogni **studente** ha accesso a un **profilo personale** che consente di **consultare le prenotazioni future**, quelle passate, quelle annullate ed eventualmente il numero totale di accessi effettuati alla biblioteca. I bibliotecari, attraverso un'interfaccia dedicata, possono **monitorare l'andamento complessivo del servizio**, visualizzando il **tasso di occupazione delle sale**, il **numero di prenotazioni giornaliere** e il **numero di prenotazioni non confermate tramite check-in**.

Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. È previsto un **sistema di notifiche** che avvisi gli studenti della conferma



della prenotazione, di eventuali annullamenti e dell'approssimarsi dell'orario prenotato. Il sistema dovrà inoltre garantire la protezione dei dati personali e una corretta gestione dei permessi e delle visibilità tra studenti e bibliotecari.

Note:

- Fascia oraria = orario

- I termini sottolineati indicano i possibili valori dell'attributo tipo della classe Area

LEGENDA:

Classe

Attributo

Funzionalità

Attore

Classe-Attore

2.2 Revisione dei requisiti

1. *Il sistema deve consentire la registrazione degli utenti.*
2. *Il sistema deve offrire all'Utente registrato una funzionalità di accesso tramite autenticazione con credenziali personali.*
3. *Di ogni Utente si vuole memorizzare nome, cognome, credenziali personali (e-mail istituzionale e password) e ruolo (studente o bibliotecario).*
4. *Di ogni Studente si vuole memorizzare il numero di matricola.*
5. *Di ogni Bibliotecario si vuole memorizzare il codice identificativo interno.*
6. *Il sistema deve offrire al Bibliotecario una funzionalità per creare e gestire Sale Studio.*
7. *Di ogni Sala Studio si vuole memorizzare nome, descrizione, numero totale di postazioni disponibili e orari di apertura giornalieri.*
 - *Ambiguità: bisogna capire se le sale sono aperte la domenica e se ci sono giorni di chiusura straordinaria*
 - *Riposta: aperta nei giorni feriali*
8. *Il sistema deve consentire al Bibliotecario di suddividere ogni Sala Studio in una o più Aree (tipologia: silenziosa, consultazione, lavoro di gruppo ecc...).*
9. *Il sistema deve consentire al Bibliotecario di visualizzare l'elenco delle prenotazioni effettuate per ciascuna Sala Studio e monitorarne lo stato.*
10. *Il sistema deve offrire agli Studenti autenticati una funzionalità per consultare le sale disponibili.*
11. *Il sistema deve permettere allo Studente di verificare le fasce orarie prenotabili per ogni giorno e per ogni sala.*
 - *Ambiguità: bisogna capire se per "Disponibilità" si intende lo stato di apertura o chiusura della Sala Studio oppure la presenza di posti disponibili*
 - *Risposta: presenza di posti disponibili*
12. *Il sistema deve consentire allo studente di effettuare una Prenotazione selezionando Sala, data e fascia oraria.*
 - *Ambiguità: bisogna capire se le fasce orarie sono predefinite dal bibliotecario (es. slot fissi di 2h: 8-10, 10-12...) o lo studente le definisce liberamente*
 - *Risposta: fasce orarie prestabilite dal bibliotecario*
13. *Il sistema deve consentire di selezionare una specifica Area ed eventualmente una specifica Postazione durante la Prenotazione.*
 - *Ambiguità: Bisogna capire se le postazioni appartengono a un'area specifica oppure le aree e le postazioni sono due dimensioni indipendenti oppure se quando la sala non è suddivisa in aree, esistono comunque le postazioni numerate; come funziona l'assegnazione della Postazione? Come per i biglietti aerei?*
 - *Risposta: sì, come i biglietti aerei. C'è sempre almeno un'Area*
14. *Il sistema deve impedire che una stessa Postazione venga assegnata contemporaneamente a più studenti nella medesima fascia oraria.*
15. *Il sistema deve aggiornare automaticamente lo stato della Prenotazione ad "attiva" una volta completata.*
16. *Di ogni Prenotazione si vuole memorizzare Sala Studio, data, fascia oraria e stato.*
17. *Il sistema deve consentire allo Studente di visualizzare, all'interno del Profilo Personale, le proprie informazioni.*
18. *Di ogni Profilo Personale si vuole memorizzare Prenotazioni future e passate, Prenotazioni annullate, il numero totale di accessi.*
19. *Il sistema deve consentire l'annullamento di una Prenotazione entro un limite temporale precedente all'inizio della fascia oraria.*



- *Ambiguità: bisogna capire come quantificare l'intervallo di tempo*
 - *Risposta: tolleranza di 6 ore*
20. *Il sistema deve rendere la Postazione nuovamente disponibile per altri utenti in caso di annullamento della prenotazione.*
21. *Il sistema deve offrire allo studente una funzionalità per effettuare il check-in tramite interfaccia grafica.*
- *Ambiguità: bisogna capire come avviene tecnicamente il check-in; tramite codice QR generato sulla postazione, geolocalizzazione, o semplice tasto sulla web-app?*
 - *Risposta: tasto di conferma sull'interfaccia*
22. *Il sistema deve consentire il check-in solo entro un intervallo di tempo limitato rispetto all'orario di inizio.*
23. *Il sistema deve aggiornare lo stato della Prenotazione a "scaduta" se in check-in non viene effettuato entro il determinato intervallo.*
- *Ambiguità: bisogna capire come quantificare l'intervallo di tempo; cosa succede allo scadere dell'intervallo? Ci sono penalità per lo studente?*
 - *Risposta: tolleranza di 10 minuti. Per le penalità bisogna modellare il comportamento.*
24. *Il sistema deve rendere nuovamente disponibile la Postazione se la Prenotazione risulta "scaduta".*
25. *Il sistema deve offrire al Bibliotecario una funzionalità per consultare, in tempo reale, lo stato delle Sale Studio (postazioni libere/occupate e prenotazioni attive per la giornata corrente).*
26. *Il sistema deve offrire al Bibliotecario l'accesso allo storico delle Prenotazioni per Sala Studio e Studente.*
27. *Il sistema deve offrire al Bibliotecario una funzionalità per monitorare il tasso di occupazione, il numero di prenotazioni giornaliere e il numero di prenotazioni non confermate.*
28. *Il sistema deve essere accessibile via web sia da desktop che da dispositivi mobili con interfaccia grafica intuitiva e responsiva.*
29. *Il sistema deve inviare le notifiche per confermare le prenotazioni, gli annullamenti e promemoria orari.*
- *Ambiguità: Bisogna capire se il sistema di notifiche è interno o esterno e il quando del promemoria*
 - *Risposta: esterno*
30. *Il sistema deve garantire la protezione dei dati personali.*
31. *Il sistema deve garantire la corretta gestione dei permessi di visibilità.*

2.3 Glossario dei termini

Termine	Descrizione	Sinonimi
Utente	Persona generica che può registrarsi al sistema, scegliendo il proprio ruolo e può autenticarsi con credenziali personali	
Credenziali	Dati personali (es. e-mail istituzionale e password) utilizzati per l'autenticazione al sistema	

Studente	Utente autenticato con ruolo di studente, identificato da matricola, che può prenotare postazioni nelle sale studio.	
Profilo Personale	Parte di ciascuno Studente in cui consultare lo storico delle prenotazioni e le prenotazioni attive (future)	
Bibliotecario	Utente autenticato con ruolo di bibliotecario, identificato da codice identificativo interno, che gestisce le sale studio e monitora le prenotazioni	
Sala Studio	Ambiente fisico gestito da un bibliotecario, caratterizzato da nome, descrizione, numero di postazioni e orari di apertura	Sala
Area	Suddivisione di una sala studio, caratterizzata da una specifica destinazione d'uso (silenziosa, consultazione, lavoro di gruppo)	
Postazione	Singolo posto a sedere all'interno di una sala studio, appartenente opzionalmente a una specifica area, prenotabile da un solo studente per fascia oraria	
Fascia Oraria	Intervallo di tempo prenotabile all'interno degli orari di apertura di una sala	Orario
Prenotazione	Atto di riservare una postazione in un'area di una sala, per una data e una fascia oraria specifiche, da parte di uno studente	
Storico Prenotazioni	Archivio delle prenotazioni passate (annullate e confermate) consultabile per sala o per studente e numero totale di accessi	
Stato della Prenotazione	Condizione corrente di una prenotazione: può essere "attiva", "annullata", "scaduta" o "confermata"	
Stato della Postazione	Condizione corrente di una postazione: può essere "libera" o "occupata"	
Check-in	Azione con cui lo studente conferma la propria presenza in sala il giorno della prenotazione, entro un intervallo di tempo limitato	
Notifica	Avviso inviato allo studente per confermare prenotazioni, segnalare annullamenti o ricordare l'approssimarsi dell'orario prenotato	

2.4 Classificazione dei requisiti

2.4.1 Requisiti funzionali

ID	Requisito	Origine (n. frase dei requisiti revisionati)
RF01	Il sistema deve consentire la registrazione degli utenti.	1
RF02	Il sistema deve offrire all'Utente registrato una funzionalità di accesso tramite autenticazione con credenziali personali	2

RF03	Il sistema deve offrire al Bibliotecario una funzionalità per creare e gestire Sale Studio	6
RF04	Il sistema deve consentire al Bibliotecario di suddividere ogni Sala Studio in una o più Aree (tipologia: silenziosa, consultazione, lavoro di gruppo etc....).	8
RF05	Il sistema deve consentire al Bibliotecario di visualizzare l'elenco delle prenotazioni effettuate per ciascuna Sala Studio e monitorarne lo stato.	9
RF06	Il sistema deve offrire agli Studenti autenticati una funzionalità per consultare le sale disponibili.	10
RF07	Il sistema deve permettere allo Studente di verificare le fasce orarie prenotabili per ogni giorno e per ogni sala.	11
RF08	Il sistema deve consentire allo studente di effettuare una Prenotazione selezionando Sala, data e fascia oraria	12
RF09	Il sistema deve consentire di selezionare una specifica Area ed eventualmente una Postazione durante la Prenotazione	13
RF10	Il sistema deve consentire allo Studente di visualizzare, all'interno del proprio Profilo Personale, le prenotazioni future, passate, annullate e il numero totale di accessi.	17
RF11	Il sistema deve consentire allo Studente l'annullamento di una Prenotazione entro un limite temporale precedente all'inizio della fascia oraria.	19
RF12	Il sistema deve offrire allo studente una funzionalità per effettuare il check-in tramite interfaccia grafica.	21
RF13	Il sistema deve offrire al Bibliotecario una funzionalità per consultare, in tempo reale, lo stato delle Sale Studio (postazioni libere/occupate e prenotazioni attive per la giornata corrente).	25
RF14	Il sistema deve offrire al Bibliotecario l'accesso allo storico delle Prenotazioni per Sala Studio e Studente.	26
RF15	Il sistema deve offrire al Bibliotecario una funzionalità per monitorare il tasso di occupazione, il numero di prenotazioni giornaliere e il numero di prenotazioni non confermate.	27
RF16	Il sistema deve inviare le notifiche per confermare le prenotazioni, gli annullamenti e promemoria orari.	29

2.4.2 Requisiti sui dati

ID	Requisito	Origine (n. frase dei requisiti revisionati)
RD01	Di ogni Utente si vuole memorizzare nome, cognome, credenziali personali (e-mail istituzionale e password) e ruolo (studente o bibliotecario).	3

RD02	Di ogni Studente si vuole memorizzare il numero di matricola.	4
RD03	Di ogni Bibliotecario si vuole memorizzare il codice identificativo interno.	5
RD04	Di ogni Sala Studio si vuole memorizzare nome, descrizione, numero totale di postazioni disponibili e orari di apertura giornalieri.	7
RD05	Di ogni Prenotazione si vuole memorizzare Sala Studio, data, fascia oraria e stato ed eventualmente l'Area o la Postazione specifica.	16
RD06	Di ogni Profilo Personale si vuole memorizzare Prenotazioni future, passate, annullate ed il numero totale di accessi.	18
RD07	Di ogni Area si vuole memorizzare la tipologia (silenziosa, consultazione, lavoro di gruppo, ...) e la Sala Studio cui appartiene.	8
RD08	Di ogni Postazione si vuole memorizzare la Sala Studio a cui appartiene e se prevista l'Area.	13 e 14
RD09	Di ogni Fascia Oraria si vuole memorizzare l'orario di inizio e l'orario di fine.	11 e 12

2.4.3 Requisiti Non Funzionali di Qualità (si può fare riferimento alle caratteristiche dello Standard ISO 25010)

ID	Categoria	Descrizione	Metrica	Valore atteso	Verifica
RQ-QUAL-01	Sicurezza [Integrità]	Il sistema deve garantire la protezione dei dati personali	% di dati personali accessibili solo previa autenticazione	100%	Test di penetrazione
RQ-QUAL-02	Sicurezza [Riservatezza]	Il sistema deve garantire una corretta gestione dei permessi di visibilità	% di tentativi di accesso a risorse non autorizzate correttamente e bloccati	100%	Test funzionali di accesso cross-ruolo (es. Studente che tenta endpoint da Bibliotecario)
RQ-QUAL-03	Performance [Comportamento temporale]	Il sistema deve offrire al Bibliotecario una funzionalità per consultare,	Tempo di risposta per il caricamento della dashboard delle sale.	≤ 2 secondi nel 95% delle richieste	Test di carico simulando un database pieno di prenotazioni (tutte le postazioni occupate)

		in tempo reale, lo stato delle Sale Studio			
RQ-QUAL-04	Portabilità [Adattabilità]	Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva.	% di browser e risoluzioni target su cui l'interfaccia è correttamente fruibile (nessuna funzionalità persa, nessuna sovrapposizione visiva)	100% dei browser principali (Chrome, Firefox, Safari, Edge) e delle risoluzioni rappresentative desktop/tablet/mobile	Test multiplatforma su matrice di browser e dispositivi, con verifica visiva e funzionale a diverse risoluzioni
RQ-QUAL-05	Usabilità [Apprendibilità]	L'interfaccia grafica deve essere intuitiva, permettendo a uno studente di prenotare o fare check-in senza formazione preventiva.	Tempo massimo per completare una prenotazione al primo utilizzo	≤ 3 minuti	Test di usabilità con un campione di studenti non istruiti sul sistema
RQ-QUAL-06	Affidabilità [Tolleranza ai guasti]	Il sistema deve impedire l'assegnazione concorrente di una stessa postazione a più studenti nella medesima fascia oraria.	Numero di "overbooking" o conflitti di prenotazione consentiti dal sistema.	0	Test di concorrenza iniettando richieste simultanee sulla stessa postazione.

2.4.4 Vincoli / Altri requisiti

ID	Requisito	Origine (n. frase dei requisiti revisionati)
----	-----------	--

Vo1	Per l'invio delle notifiche di conferma, annullamento e promemoria orario, deve essere integrato un servizio di messaggistica esterno al sistema.	29
Vo2	Il sistema deve essere sviluppato come applicazione web, senza richiedere l'installazione di software client dedicato nei dispositivi degli utenti.	28
Vo3	Il sistema deve essere conforme alle normative vigenti in materia di privacy (GDPR) per garantire la protezione dei dati personali degli studenti e personale.	30
Vo4	Ogni sala studio deve avere almeno un'area ed ogni area deve essere associata ad almeno una postazione.	13
Vo5	Ogni Fascia Oraria prenotabile deve essere compresa negli orari di apertura giornalieri della Sala Studio cui si riferisce.	12

2.5 Modellazione dei casi d'uso

2.5.1 Attori e casi d'uso

Attori Primari:

- Utente
- Studente
- Bibliotecario
- Tempo

Attori Secondari:

- ServizioNotifiche

Casi d'uso:

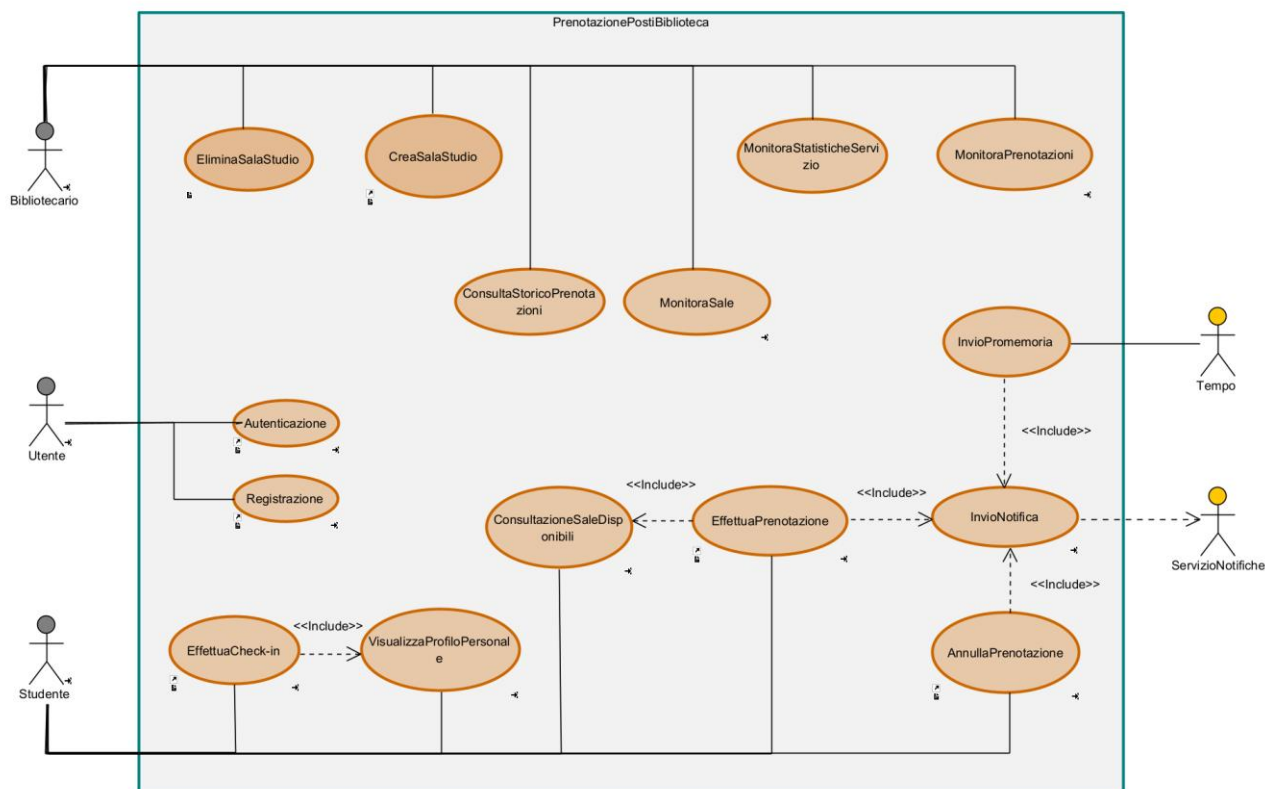
- **UC1:** Registrazione
- **UC2:** Autenticazione
- **UC3:** CreaSalaStudio
- **UC4:** EliminaSalaStudio
- **UC5:** MonitoraPrenotazioni
- **UC6:** ConsultazioneSaleDisponibili
- **UC7:** EffettuaPrenotazione
- **UC8:** VisualizzaProfiloPersonale
- **UC9:** AnnullaPrenotazione
- **UC10:** EffettuaCheck-in
- **UC11:** MonitoraSale
- **UC12:** ConsultaStoricoPrenotazioni
- **UC13:** MonitoraStatisticheServizio
- **UC14:** InvioPromemoria

Casi d'uso di inclusione:

- **UC15:** InvioNotifica

Caso d'uso	Attori Primari	Attori Secondari	Incl. / Ext.	Requisiti corrispondenti
UC1: Registrazione	Utente	-	-	RF01
UC2: Autenticazione	Utente	-	-	RF02
UC3: CreaSalaStudio	Bibliotecario	-	-	RF03, RF04
UC4: EliminaSalaStudio	Bibliotecario	-	-	RF03
UC5: MonitoraPrenotazioni	Bibliotecario	-	-	RF05
UC6: ConsultazioneSaleDisponibili	Studente	-	Incluso in EffettuaPrenotazione	RF06, RF07
UC7: EffettuaPrenotazione	Studente	-	Include ConsultaSaleStudio, InvioNotifica	RF08, RF09
UC8: VisualizzaProfiloPersonale	Studente	-	-	RF10
UC9: AnnullaPrenotazione	Studente	-	Include InvioNotifica	RF11
UC10: EffettuaCheck-in	Studente	-	-	RF12
UC11: MonitoraSale	Bibliotecario	-	-	RF13
UC12: ConsultaStoricoPrenotazioni	Bibliotecario	-	-	RF14
UC13: MonitoraStatisticheServizio	Bibliotecario	-	-	RF15
UC14: InvioNotifica	-	ServizioNotifiche	Incluso in EffettuaPrenotazione, AnnullaPrenotazione, InvioPromemoria	RF16
UC15: InvioPromemoria	Tempo	ServizioNotifiche	Include InvioNotifica	RF16

2.5.2 Diagramma dei casi d'uso



2.5.3 Scenari

Caso d'uso:	Registrazione
Attore primario	Utente
Attore secondario	-
Descrizione	Un Utente non registrato richiede di registrarsi nel sistema delle prenotazioni delle postazioni della biblioteca.
Pre-Condizioni	L'Utente non registrato con e-mail istituzionale.
Sequenza di eventi principale	1. Il caso d'uso inizia quando l'Utente accede alla sezione dedicata alla registrazione dall'interfaccia web. 2. L'Utente specifica ruolo, nome, cognome, e-mail istituzionale e sceglie una password. 3. Il sistema verifica l'esistenza del ruolo selezionato. 4. if Il ruolo selezionato è "Studente": 4.1. L'Utente inserisce il numero di matricola. 5. else if Il ruolo selezionato è "Bibliotecario": 5.1. L'Utente inserisce il codice identificativo interno. end if 6. Il sistema verifica la correttezza dei dati e l'univocità dell'account. 7. Il sistema crea il nuovo profilo utente e memorizza i dati.
Post-Condizioni	L'Utente viene registrato nel sistema e può autenticarsi tramite le proprie credenziali.

Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 6, se l'e-mail istituzionale o il codice/matricola risultano già presenti nel sistema il sistema restituisce un messaggio di errore e invita l'utente a correggere i dati.

Caso d'uso:	Autenticazione
Attore primario	Utente
Attore secondario	-
Descrizione	Un Utente registrato richiede di accedere al sistema delle prenotazioni delle Postazioni della biblioteca tramite le proprie credenziali personali.
Pre-Condizioni	L'Utente deve aver effettuato la <i>Registrazione</i> nel sistema e non deve aver ancora effettuato l'autenticazione.
Sequenza di eventi principale	1. Il caso d'uso inizia quando l'Utente non autenticato richiede di autenticarsi nel sistema delle prenotazioni delle postazioni della biblioteca. 2. L'Utente inserisce la propria e-mail istituzionale e la password scelte in fase di registrazione. 3. Il sistema verifica la corrispondenza tra e-mail istituzionale e password memorizzate.
Post-Condizioni	L'Utente risulta autenticato nel sistema e può accedere alle funzionalità consentite dal proprio ruolo.
Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 3, se l'e-mail istituzionale non risulta presente nel sistema oppure la password non corrisponde, il sistema mostra un messaggio di errore e impedisce l'accesso.

Caso d'uso:	EffettuaPrenotazione
Attore primario	Studente
Attore secondario	ServizioNotifiche
Descrizione	Uno Studente prenota una postazione in una sala studio, specificando data, fascia oraria e area. Lo Studente può scegliere una postazione specificata oppure richiedere l'assegnazione automatica da parte del sistema
Pre-Condizioni	-
Sequenza di eventi principale	1. <<include>> ConsultazioneSaleDisponibili: il sistema mostra le sale studio disponibili con le rispettive aree, postazioni e fasce orarie prenotabili. Lo Studente individua la sala, la data, la fascia oraria e l'area desiderate. 2. Il caso d'uso inizia quando lo Studente, dopo aver individuato sala, data, fascia oraria ed opzionalmente l'area desiderata, avvia la prenotazione. 3. if lo Studente ha indicato un'area specifica: 3.1. Il sistema verifica che nell'area e nella fascia oraria della data selezionate esistano postazioni libere. 4. else:

	4.1. Il sistema verifica che nella sala studio e nella fascia oraria della data selezionate esistano postazioni libere. end if 5. Il sistema mostra le postazioni disponibili. 6. Lo Studente seleziona una postazione specifica oppure richiede l'assegnazione automatica. 7. if lo Studente ha richiesto l'assegnazione automatica: 7.1. Il sistema seleziona la prima postazione libera disponibile nell'area scelta per la specifica fascia oraria. 8. else: 8.1. Il sistema verifica che la postazione selezionata sia ancora disponibile. end if 9. Il sistema registra la prenotazione e ne imposta lo stato ad "attiva". 10. Il sistema aggiorna il profilo personale dello studente 11. <<include>> InvioNotifica: il sistema invia allo Studente una notifica di conferma della prenotazione.
Post-Condizioni	La prenotazione è memorizzata in stato "attiva" ed appare nel profilo personale dello Studente. La postazione assegnata risulta occupata per la fascia oraria e data specificata.
Casi d'uso correlati	<i>ConsultazioneSaleDisponibili, InvioNotifica.</i>
Sequenza di eventi alternativi	Al punto 3.1, se nell'area selezionata non esistono postazioni libere per la fascia oraria scelta, il sistema segnala l'indisponibilità ed il caso d'uso termina. Al punto 4.1, se nella sala studio selezionata non esistono postazioni libere per la fascia oraria scelta, il sistema segnala l'indisponibilità ed il caso d'uso termina. Al punto 8.1, se la postazione selezionata risulta nel frattempo occupata, il sistema segnala l'indisponibilità e ritorna al punto 7 per consentire allo Studente di selezionare una postazione diversa o richiedere l'assegnazione automatica. Al punto 11, se il sistema non riesce a recapitare la notifica di conferma, la prenotazione rimane comunque registrata con stato "attiva" e il sistema segnala allo Studente il mancato invio della notifica. Il caso d'uso termina.

Caso d'uso:	AnnullaPrenotazione
Attore primario	Studente
Attore secondario	ServizioNotifiche
Descrizione	Lo Studente annulla una prenotazione attiva entro il limite temporale consentito prima dell'inizio della fascia prenotata.
Pre-Condizioni	Lo Studente deve possedere almeno una prenotazione attiva.
Sequenza di eventi principale	1. Il caso d'uso inizia quando lo Studente chiede di annullare una prenotazione attiva. 2. Il sistema verifica che l'annullamento avvenga almeno 6 ore prima dell'inizio della fascia oraria prenotata. 3. Il sistema imposta allo stato "annullata" la prenotazione selezionata e il sistema rende nuovamente disponibile la postazione. 4. <<include>> InvioNotifica: il sistema invia una notifica di conferma dell'annullamento allo Studente. 5. Il sistema aggiorna il profilo personale dello Studente.

Post-Condizioni	La prenotazione risulta annullata, la postazione torna disponibile e il profilo personale risulta aggiornato.
Casi d'uso correlati	<i>InvioNotifica</i>
Sequenza di eventi alternativi	Al punto 2, se mancano meno di 6 ore all'inizio della fascia oraria prenotata, il sistema impedisce l'annullamento e informa lo Studente che il limite temporale è stato superato.

Caso d'uso:	CreaSalaStudio
Attore primario	Bibliotecario
Attore secondario	-
Descrizione	Il Bibliotecario crea una nuova sala studio specificando tutte le informazioni necessarie e definendo le relative aree
Pre-Condizioni	-
Sequenza di eventi principale	1. Il caso d'uso inizia quando il Bibliotecario accede alla sezione per la creazione di una nuova sala studio. 2. Il Bibliotecario inserisce nome, descrizione, numero totale di postazioni da creare, orari di apertura giornalieri e numero di aree da creare. 3. Il sistema verifica la validità dei dati inseriti. 4. Il sistema inizializza la nuova Sala Studio con le rispettive postazioni. 5. while ci sono ancora aree da creare 5.1. Il Bibliotecario specifica la tipologia dell'area ed assegna un numero specifico di postazioni all'area. 5.2. Il sistema verifica che l'area contenga almeno una postazione e che il numero di postazioni totale nelle varie aree non superi il limite massimo disponibile per la sala studio. 5.3. Il sistema crea l'area e l'associa alla sala studio corrente. end while 6. Il sistema memorizza la nuova sala studio con le sue relative aree e postazioni.
Post-Condizioni	La nuova Sala Studio è memorizzata nel sistema con le sue relative aree e postazioni ed è visibile agli studenti.
Casi d'uso correlati	
Sequenza di eventi alternativi	Al punto 3, se i dati inseriti non sono validi o sono incompleti, il sistema mostra un messaggio di errore e richiede la correzione. Al punto 5.2, se la somma delle postazioni delle aree eccede il totale della sala, il sistema deve impedire l'inserimento dell'Area.

Caso d'uso:	EliminaSalaStudio
Attore primario	Bibliotecario
Attore secondario	-
Descrizione	Il Bibliotecario elimina una sala studio esistente e tutte le relative aree.
Pre-Condizioni	Non devono esserci Prenotazioni attive per la sala scelta.
Sequenza di eventi principale	1. Il caso d'uso inizia quando il Bibliotecario seleziona la sala da eliminare. 2. Il sistema verifica l'esistenza della sala scelta. 3. Il sistema elimina tutte le aree e postazioni associate alla Sala Studio 4. Il sistema elimina la Sala Studio.

Post-Condizioni	L'area non è più presente nel sistema e non è più visibile agli studenti
Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 2, se la sala selezionata non esiste, il sistema segnala l'incongruenza ed invita a scegliere nuovamente una sala.

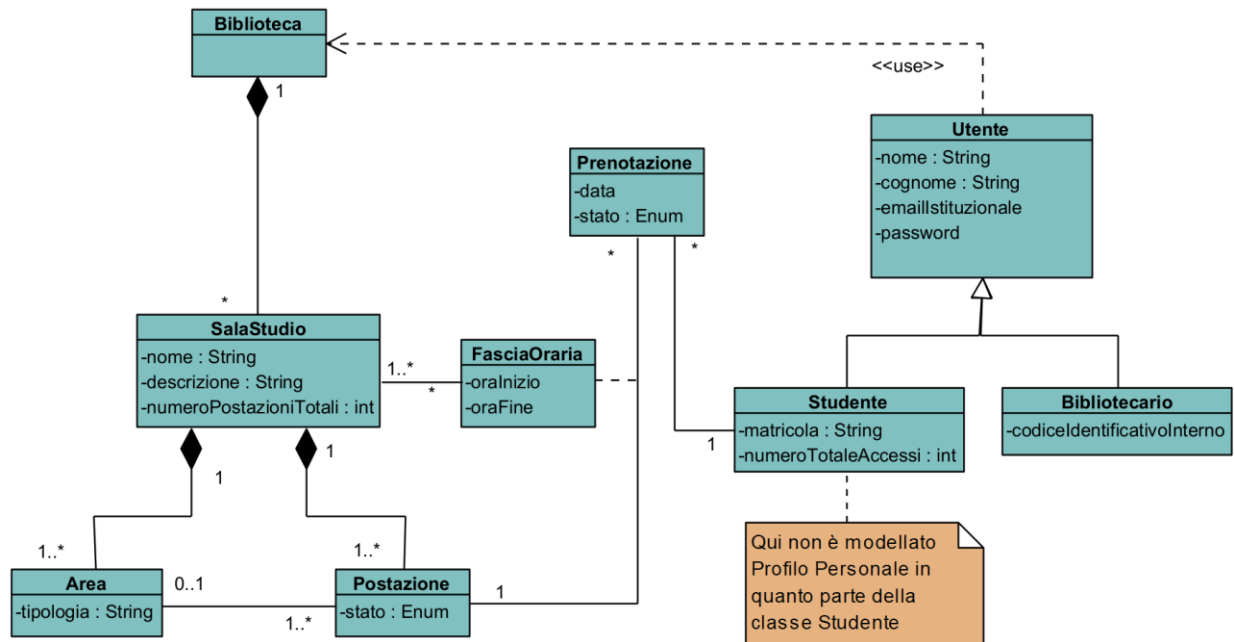
Caso d'uso:	EffettuaCheck-in
Attore primario	Studente
Attore secondario	-
Descrizione	Nel giorno della prenotazione, lo Studente conferma la propria presenza effettuando il check-in della Prenotazione attiva tramite interfaccia web.
Pre-Condizioni	Lo studente deve avere una Prenotazione attiva.
Sequenza di eventi principale	1. Il caso d'uso inizia quando lo Studente chiede di fare il check-in di una prenotazione attiva nella giornata corrente. 2. Il sistema verifica che la prenotazione abbia scadenza nella giornata corrente. 3. Il sistema verifica che l'operazione sia avvenuta all'interno dell'intervallo prestabilito. 4. Il sistema aggiorna lo stato della Prenotazione a "confermata".
Post-Condizioni	La prenotazione risulta nello stato "confermata" e la postazione rimane assegnata allo Studente per l'intera fascia oraria.
Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 2, se la prenotazione non ha scadenza nella giornata corrente, il sistema segnala l'incongruenza e il caso d'uso termina.

Caso d'uso:	ConsultazioneSaleDisponibili
Attore primario	Studente
Attore secondario	-
Descrizione	Lo Studente consulta le Sale Studio disponibili per una determinata data, visualizzando le fasce orarie prenotabili e, se richiesto, il dettaglio delle Aree e delle Postazioni libere associate.
Pre-Condizioni	Nel sistema deve essere presente almeno una Sala Studio.
Sequenza di eventi principale	1. Il caso d'uso inizia quando lo Studente chiede di consultare la disponibilità di una Sala Studio per una determinata data. 2. Il sistema verifica che la data selezionata rientri nei giorni di apertura previsti. 3. Il sistema verifica che vi siano Sale Studio disponibili per quella data. 4. Il sistema mostra allo Studente le Sale Studio disponibili e, per ciascuna di esse, le fasce orarie prenotabili. 5. Lo Studente seleziona una Sala Studio e una fascia oraria prenotabile per visualizzarne il dettaglio. 6. Per la Sala Studio selezionata e la fascia oraria, il sistema verifica se esiste almeno una Postazione libera nella data selezionata

	7. Il sistema mostra le Postazioni libere della Sala Studio selezionata (dunque anche delle relative Aree) nella data e nella fascia oraria indicate.
Post-Condizioni	Lo Studente visualizza le Sale Studio disponibili, le relative fasce orarie prenotabili e, se richiesto, le Aree e le Postazioni libere.
Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 3, se la data selezionata non è valida oppure non rientra nei giorni di apertura, il sistema mostra un messaggio di errore e invita lo Studente a selezionare una nuova data. Al punto 4, se non sono presenti Sale Studio disponibili, il sistema mostra un messaggio d'errore e il caso d'uso termina. Al punto 5, se la Sala Studio non presenta fasce orarie prenotabili per la data selezionata, il sistema informa lo Studente che non sono presenti disponibilità. Al punto 7, se non risultano disponibili Postazioni per la Sala Studio e la fascia oraria selezionate, il sistema segnala l'indisponibilità e invita lo Studente a selezionare un'altra fascia oraria o un'altra Sala Studio.

Caso d'uso:	VisualizzaProfiloPersonale
Attore primario	Studente
Attore secondario	-
Descrizione	Lo Studente consulta, all'interno del proprio Profilo Personale, le Prenotazioni future, passate e annullate, visualizzando le relative informazioni di Sala Studio, data, fascia oraria e stato, oltre al numero totale di accessi effettuati alla biblioteca.
Pre-Condizioni	Deve esistere un Profilo Personale associato allo Studente.
Sequenza di eventi principale	1. Il caso d'uso inizia quando lo Studente chiede di visualizzare il Profilo Personale. 2. Il sistema mostra allo Studente il Profilo Personale contenente: 2.1. Prenotazioni future; 2.2. Prenotazioni passate; 2.3. Prenotazioni annullate; 2.4. numero totale di accessi effettuati. 3. Per ciascuna Prenotazione, il sistema mostra Sala Studio, data, fascia oraria e stato.
Post-Condizioni	
Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 2, se lo Studente non ha ancora effettuato alcuna Prenotazione, il sistema mostra il Profilo Personale con gli elenchi delle Prenotazioni vuoti e il numero totale di accessi pari a zero.

2.6 Diagramma delle classi



2.6.1 Responsabilità

A seguire, una tabella che riassume alcune responsabilità:

RESPONSABILITÀ	CLASSE
<i>Registrazione</i>	Biblioteca
<i>Autenticazione</i>	Biblioteca
<i>CreaSalaStudio</i>	Biblioteca
<i>EliminaSalaStudio</i>	Biblioteca
<i>MonitoraPrenotazioni</i>	Biblioteca
<i>ConsultaSaleDisponibili</i>	Biblioteca
<i>EffettuaPrenotazione</i>	Biblioteca
<i>VisualizzaProfiloPersonale</i>	Studente
<i>AnnullaPrenotazione</i>	Prenotazione
<i>EffettuaCheck-in</i>	Prenotazione
<i>MonitoraSale</i>	Biblioteca
<i>MonitoraStatisticheServizio</i>	Biblioteca
<i>ConsultaStoricoPrenotazioni</i>	Studente

- Registrazione* e *Autenticazione* sono responsabilità di **Biblioteca**, in quanto `<<information Expert>>` di Utenti.

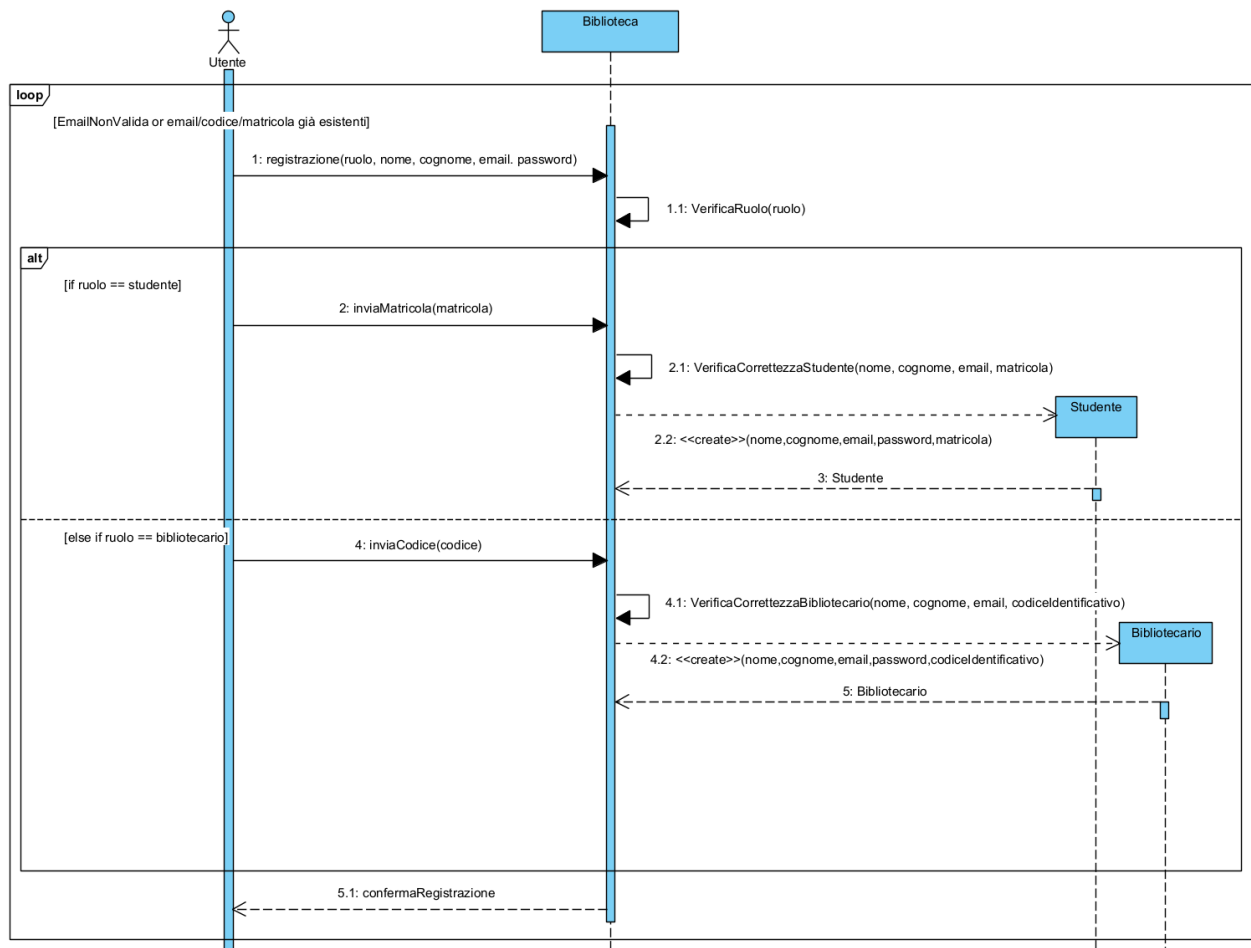
- *CreaSalaStudio* è responsabilità di **Biblioteca**, in quanto <<creator >> dell'oggetto SalaStudio.
- *EliminaSalaStudio* è responsabilità di **Biblioteca**, in quanto <<information expert >> dell'oggetto SalaStudio.
- *MonitoraPrenotazioni* e *ConsultaSaleDisponibili* sono responsabilità di **Biblioteca**, in quanto <<information expert>> di Prenotazione e Sale Studio.
- *Effettua Prenotazione* è assegnata a **Biblioteca**, poiché agisce come <<creator>> della classe Prenotazione.
- *Annulla Prenotazione* ed *EffettuaCheck-in* sono responsabilità di **Prenotazione**, in quanto <<information expert>> del proprio stato e dei vincoli temporali.
- *ConsultaStoricoPrenotazioni* è responsabilità di **Studiante**, in quanto <<information expert>> di Prenotazioni.

2.7 Diagrammi di sequenza

2.7.1 Registrazione

La creazione del suddetto sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso **Registrazione Utente**, ha evidenziato la necessità di definire alcuni metodi:

- Nella classe **Biblioteca** il metodo privato **verificaRuolo()**, necessario per controllare il ruolo selezionato dall'utente. Tale metodo permette di distinguere il flusso di registrazione in base alla tipologia di utente: **Studente** oppure **Bibliotecario**.
- Nella classe **Biblioteca** il metodo privato **verificaCorrettezzaStudente()** e **verificaCorrettezzaBibliotecario()**, necessari per validare i dati inseriti prima della creazione dell'oggetto corrispondente. Nel caso dello studente, il metodo verifica la correttezza di nome, cognome, e-mail, password e matricola; nel caso del bibliotecario, verifica nome, cognome, e-mail, password e codice identificativo.

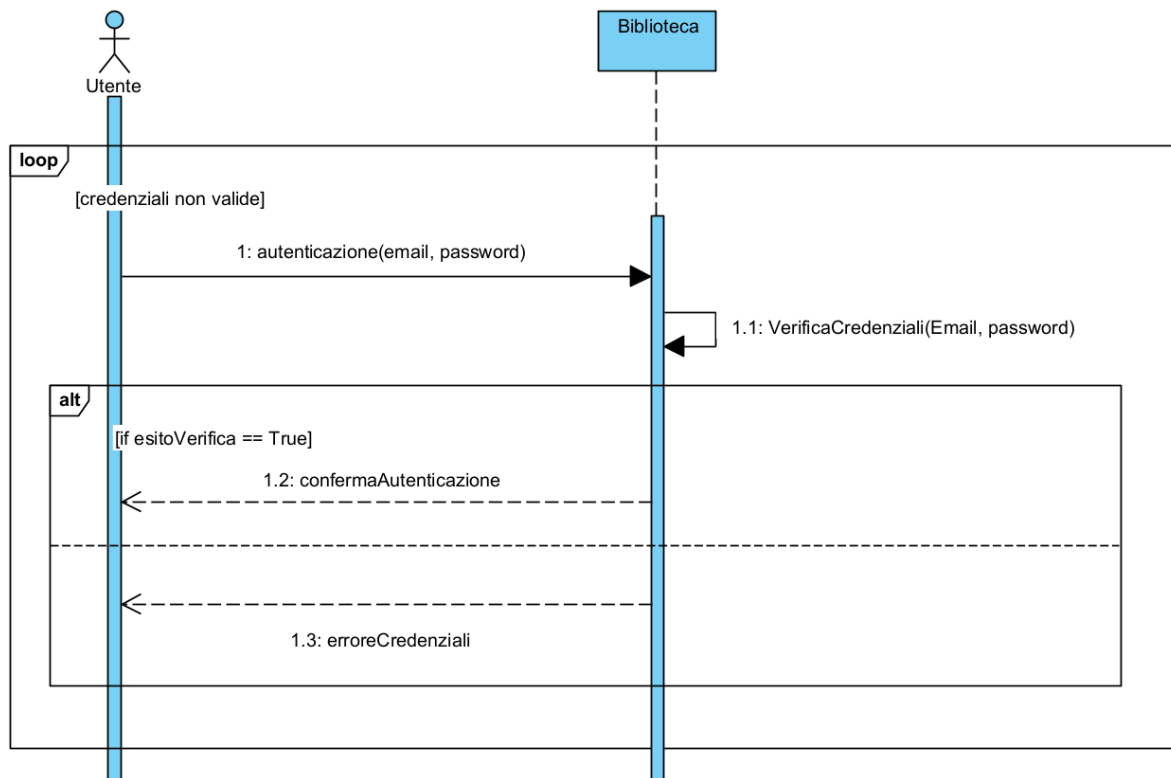


2.7.2 Autenticazione

La creazione del suddetto sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso **Autenticazione**, ha evidenziato la necessità di definire alcuni metodi:

- Nella classe **Biblioteca** il metodo privato **verificaCredenziali(email, password)**, utilizzato solo dopo che il formato delle credenziali è risultato valido.

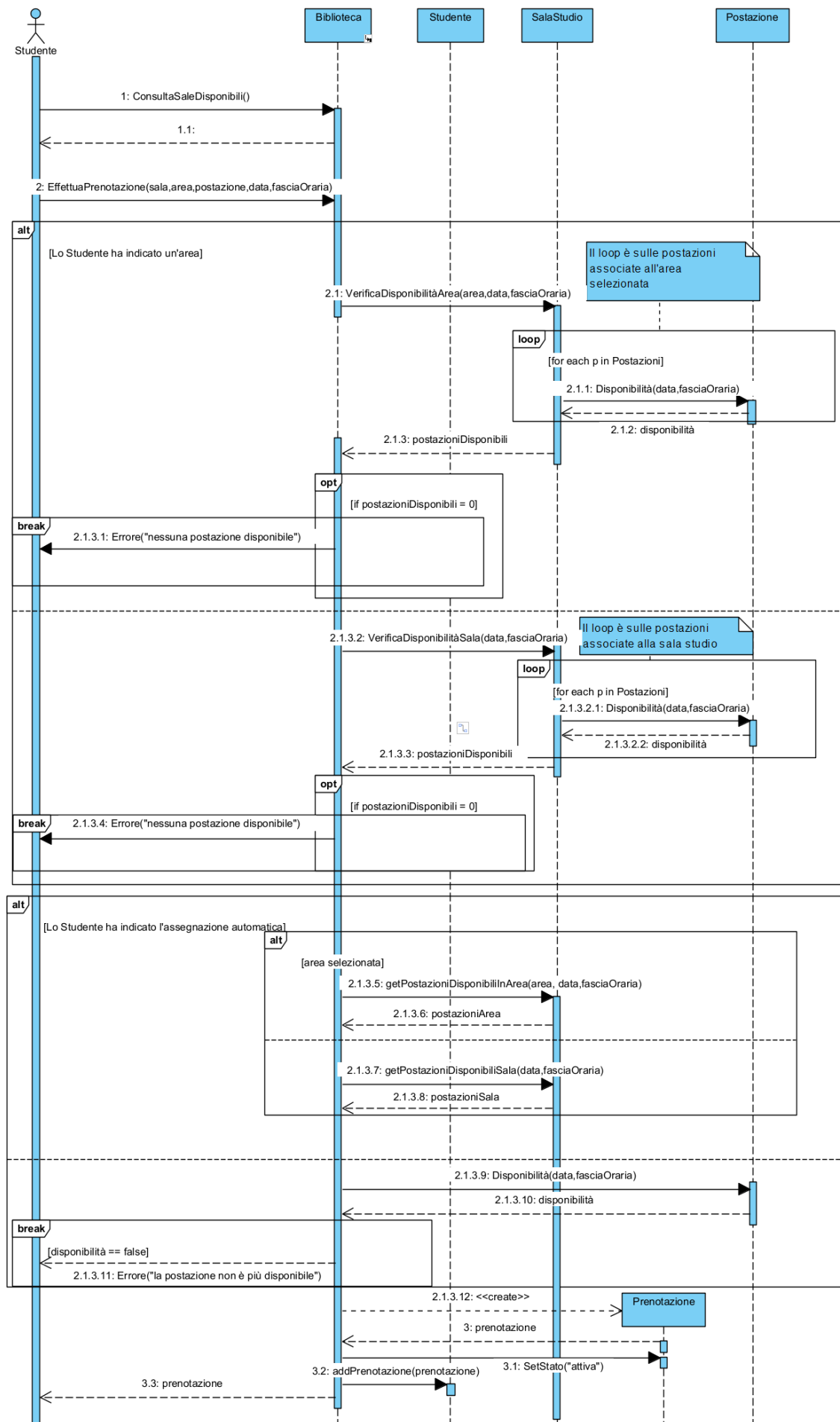




2.7.3 Effettua Prenotazione

La creazione del suddetto sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso *EffettuaPrenotazione*, ha evidenziato la necessità di definire alcuni metodi:

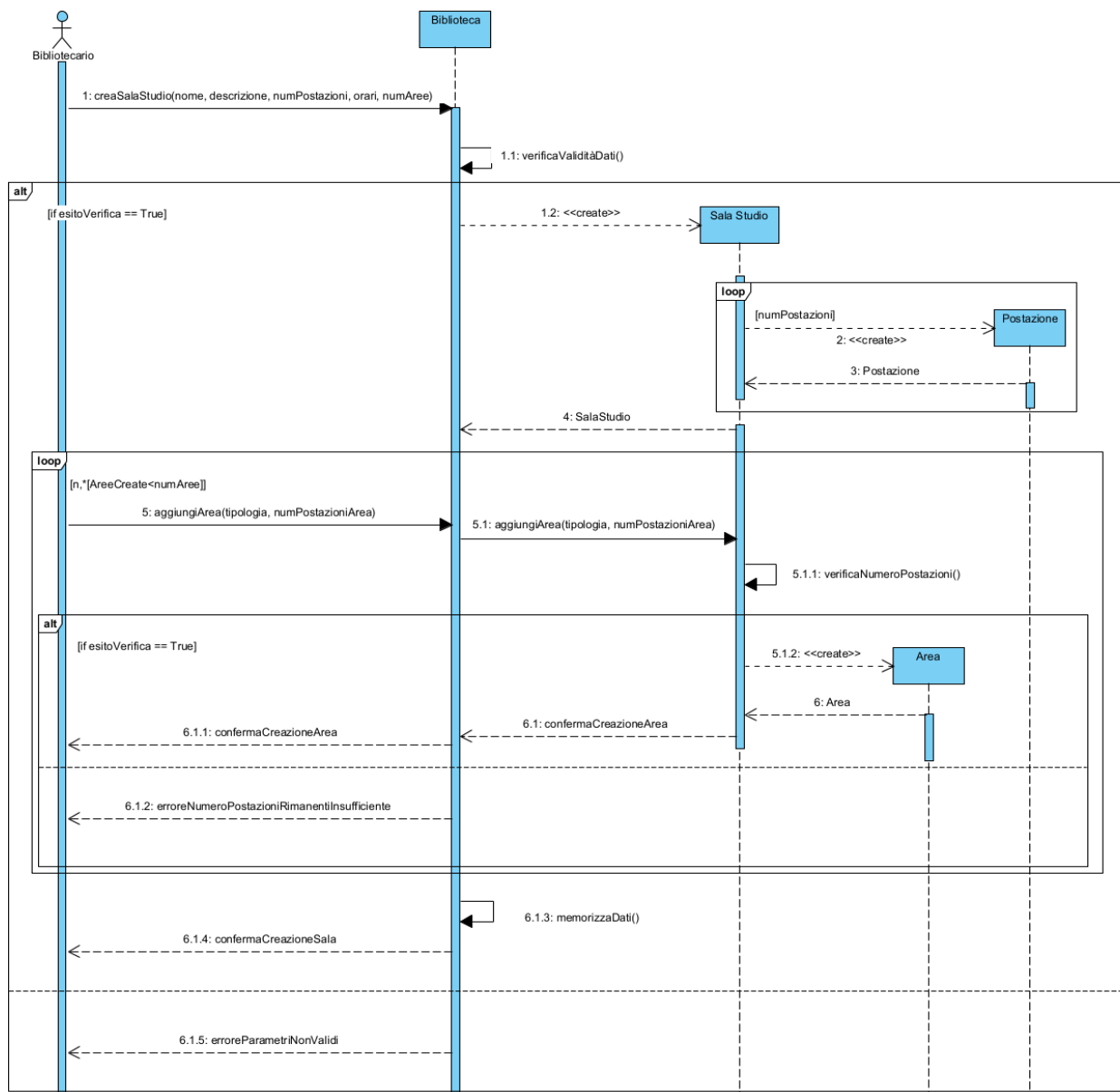
- Nella classe **SalaStudio** è necessario il metodo pubblico **VerificaDisponibilitàArea(area, data, fasciaOraria)**, che itera sulle postazioni della sala studio che sono associate all'area indicata e raccoglie quelle libere per la data e la fascia oraria richieste, restituendo l'insieme delle postazioni disponibili.
- Nella classe **SalaStudio** è necessario il metodo pubblico **VerificaDisponibilitàSala(data, fasciaOraria)**, analogo al precedente ma applicato alle sole postazioni contenute nella sala studio, utilizzato quando lo studente non ha indicato alcuna preferenza di area.
- Nella classe **Postazione** è necessario il metodo pubblico **Disponibilità(data, fasciaOraria)**, invocato in ogni iterazione del frammento loop per verificare se la singola postazione risulta "libera" nella data e nella fascia oraria indicate.
- Nella classe **Studente** il metodo pubblico **addPrenotazione(prenotazione)**, che aggiunge la prenotazione appena creata alla collezione delle prenotazioni associate allo studente.
- Nella classe **SalaStudio** il metodo pubblico **getPostazioneDisponibiliInArea(idArea, data, fasciaOraria)**, che permette di ottenere tutte le postazioni disponibili all'interno della specifica area, le quali serviranno per la scelta casuale della postazione.
- Nella classe **SalaStudio** il metodo pubblico **getPostazioniDisponibiliSala(data,fasciaOraria)**, che permette di ottenere tutte le postazioni disponibili all'interno dell'interno della sala studio non associate a nessuna area.



2.7.4 Crea Sala Studio

La creazione del suddetto sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso *CreaSalaStudio*, ha evidenziato la necessità di definire alcuni metodi:

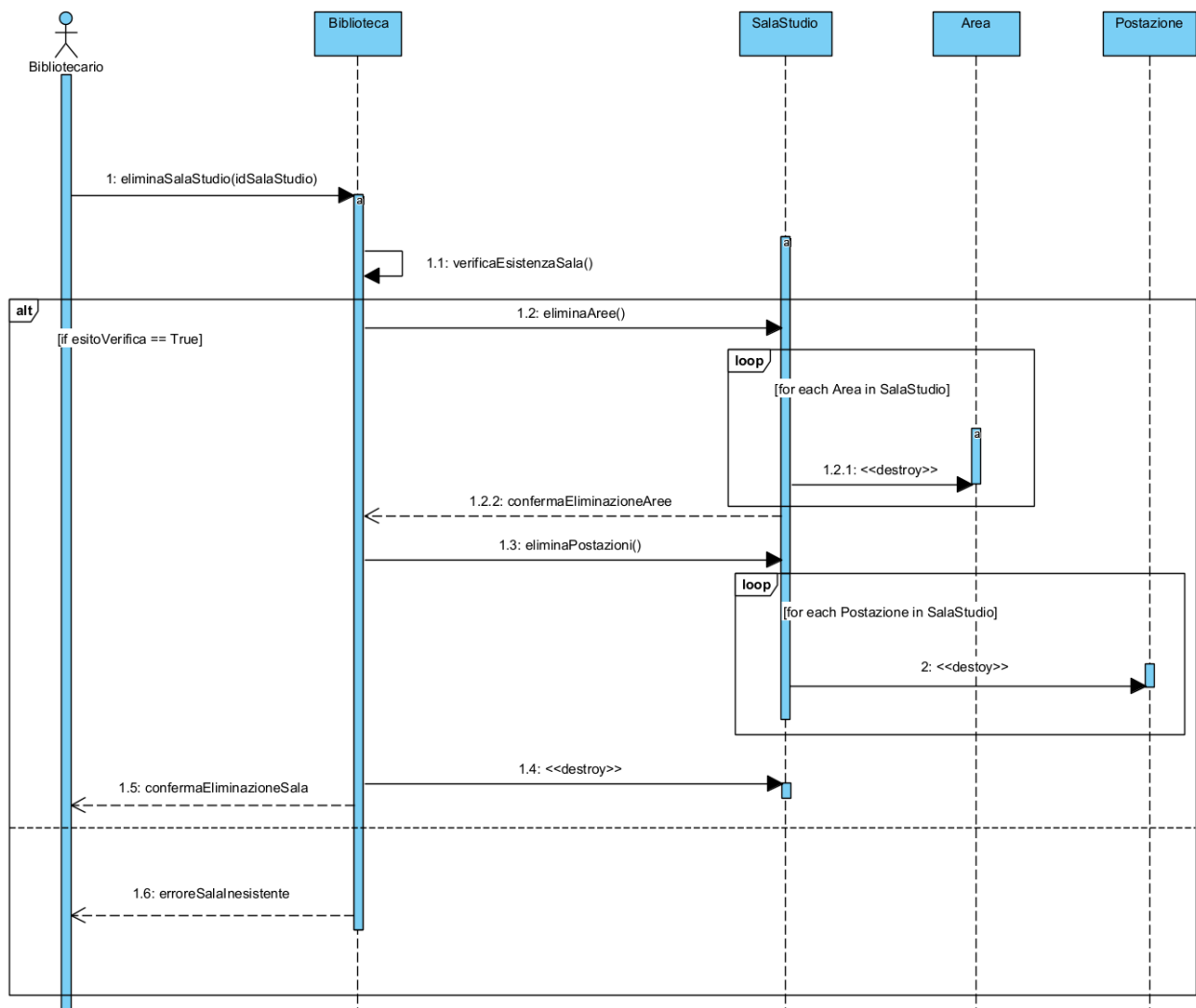
- Nella classe **Biblioteca** il metodo privato **verificaValiditaDati()**, per validare i parametri della sala (nome, orari, capienza, numeroPostazioni, NumeroAree) prima di procedere alla creazione dell'oggetto **SalaStudio**.
- Nella classe **SalaStudio** il metodo privato **verificaNumeroPostazioni()**. Tale operazione assicura che il numero di postazioni assegnate alle singole aree non ecceda mai la capacità totale definita per la sala stessa.
- Nella classe **SalaStudio** il metodo **aggiungiArea(tipologia, numPostazioniArea)** in modo da rendere possibile l'aggiunta di ulteriori Aree.



2.7.5 Elimina Sala Studio

Lo sviluppo del diagramma di sequenza per il caso d'uso *EliminaSalaStudio* ha permesso di individuare tre metodi necessari:

- Il metodo **verificaEsistenzaSala()** all'interno della classe **Biblioteca**. Questo controllo permette al sistema di gestire scenari in cui l'identificativo fornito non corrisponda a una sala esistente, prevenendo errori di esecuzione.
- I metodi **eliminaAree()** ed **eliminaPostazione()** che, tramite un ciclo iterativo, provvedono alla distruzione (<>) di tutte le rispettive istanze delle classi **Area** e **Postazione** collegate alla sala, prima di procedere alla rimozione definitiva dell'oggetto **SalaStudio** dal sistema.

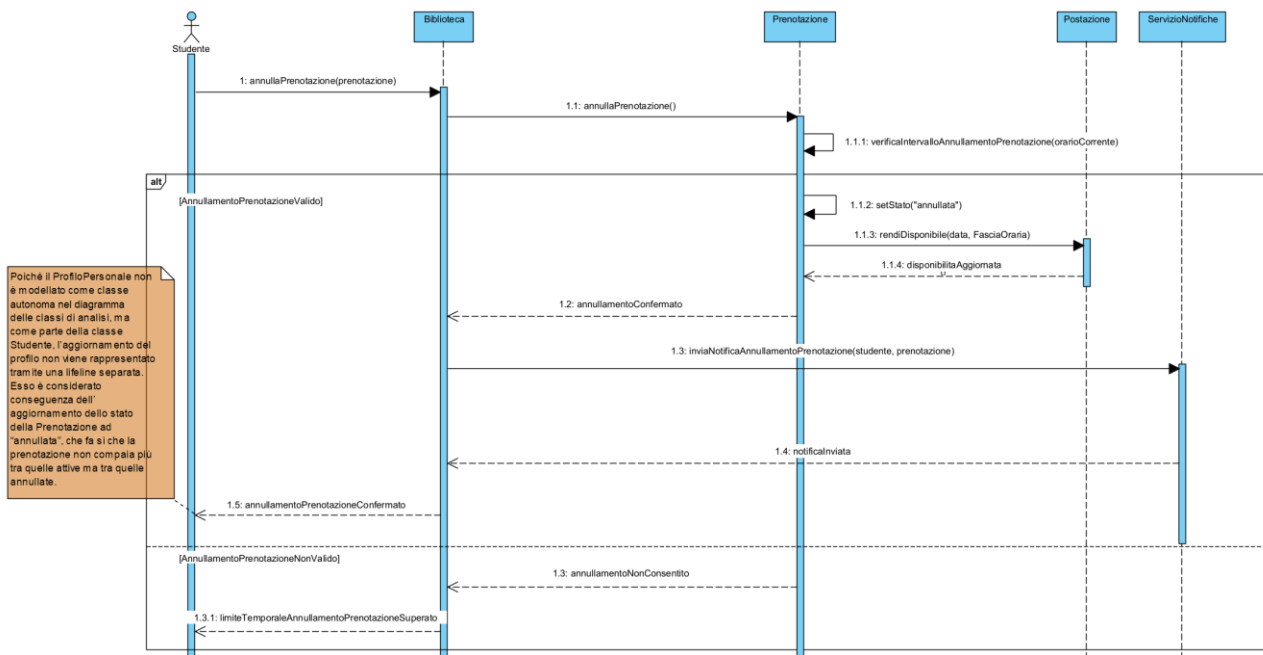


2.7.6 Annulla Prenotazione

La creazione del seguente sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso **AnnullaPrenotazione**, ha evidenziato la necessità di definire alcuni metodi distribuiti tra le classi coinvolte, in modo coerente con le responsabilità individuate nella sezione 2.6.1.

- Il metodo privato **verificaIntervalloAnnullamentoPrenotazione(orarioCorrente)**, per verificare che l'annullamento avvenga entro il limite temporale consentito.

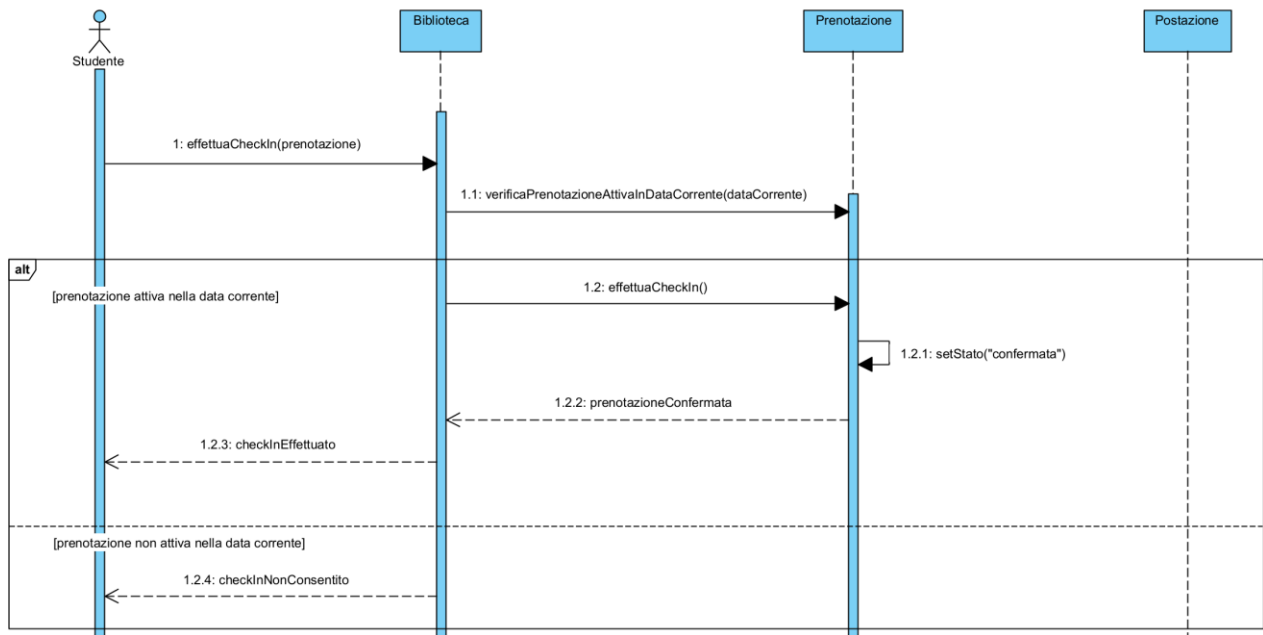
- Se l'annullamento è valido, la classe **Prenotazione** invoca **setStato("annullata")**, aggiornando il proprio stato. Successivamente viene invocato sulla classe **Postazione** il metodo **rendiDisponibile(data, fasciaOraria)**, così da rendere nuovamente disponibile la postazione associata alla prenotazione annullata.
- È necessario il metodo **inviaNotificaAnnullamentoPrenotazione(studente, prenotazione)**, che richiede al **ServizioNotifiche** l'invio della notifica di conferma dell'annullamento.
- Poiché il **ProfiloPersonale** non è modellato come classe autonoma nel diagramma delle classi di analisi, ma come parte della classe **Studente**, il suo aggiornamento non viene rappresentato tramite una lifeline separata. Esso è considerato conseguenza dell'aggiornamento dello stato della Prenotazione ad "annullata", per cui la prenotazione non compare più tra quelle attive ma tra quelle annullate.



2.7.7 Effettua Check-in

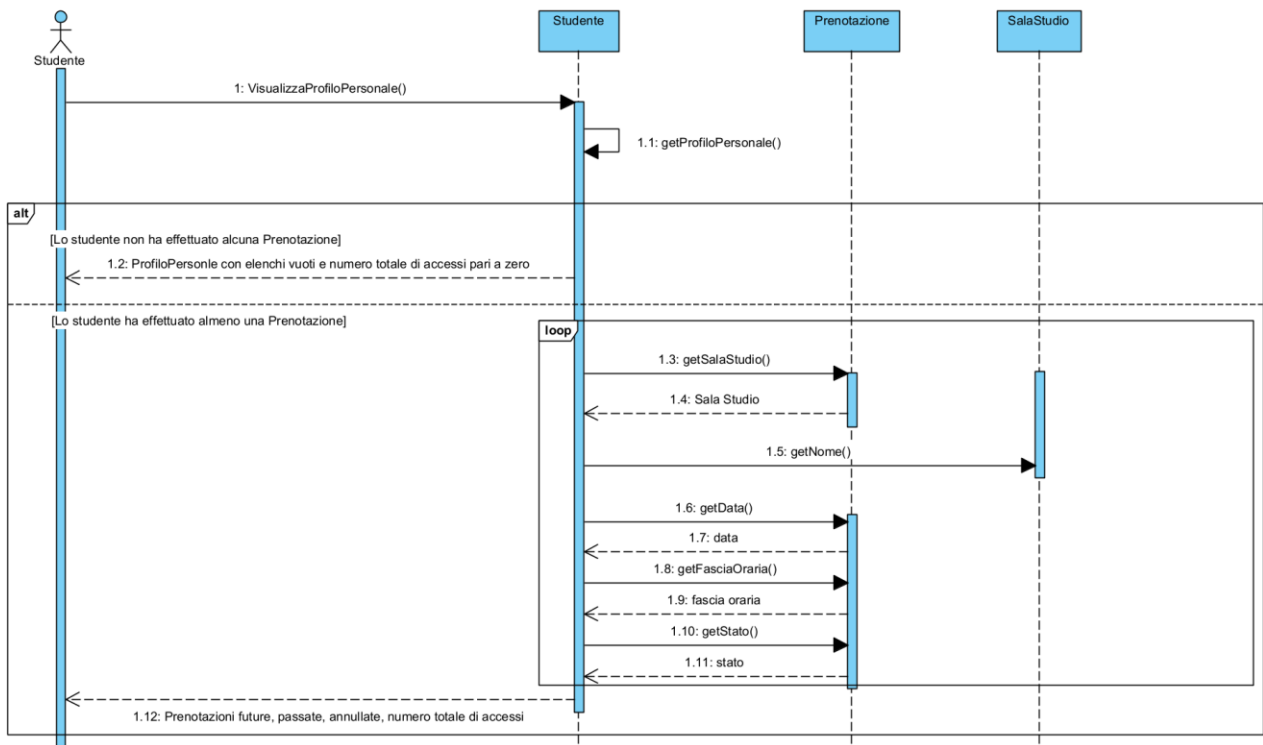
La creazione del seguente sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso **EffettuaCheck-in**, ha evidenziato la necessità di definire alcuni metodi distribuiti tra le classi coinvolte, in modo coerente con le rispettive responsabilità individuate nella sezione 2.6.1. In particolare, la responsabilità principale è assegnata alla classe **Prenotazione**, in quanto *Information Expert* del proprio stato, della data e dei vincoli temporali relativi al check-in.

- Nella classe **Prenotazione** è stato introdotto il metodo **verificaPrenotazioneAttivaInDataCorrente(dataCorrente)**, che consente di controllare che la prenotazione sia attiva e riferita alla giornata corrente, condizione necessaria per poter procedere con il check-in.
- Se l'intervallo è valido, la Prenotazione invoca **setStato("confermata")**, aggiornando il proprio stato a confermata.



2.7.8 VisualizzaProfiloPersonale

La creazione del suddetto sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso **VisualizzaProfiloPersonale**, non ha evidenziato la necessità di introdurre nuovi metodi funzionali.

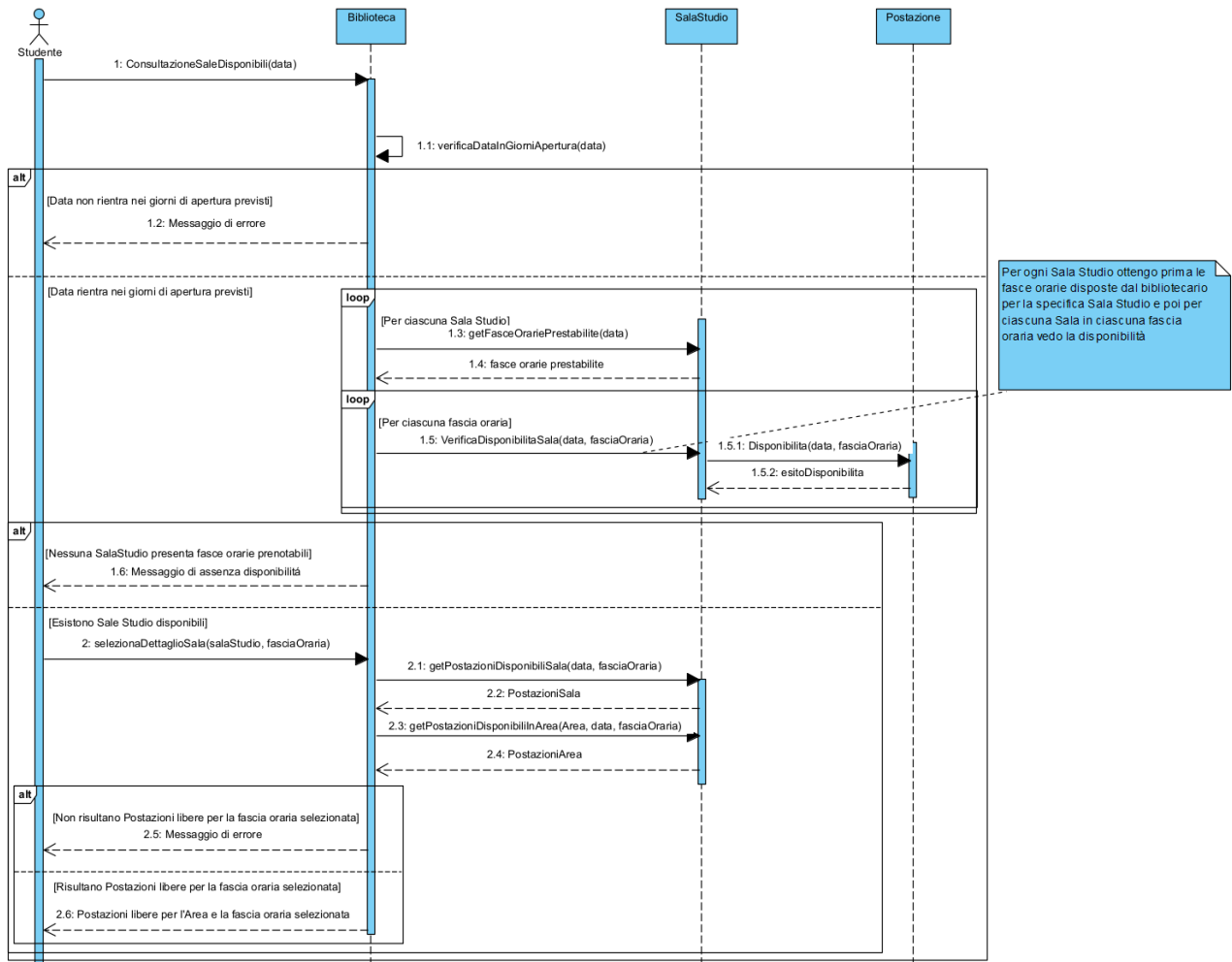


2.7.9 ConsultazioneSaleDisponibili

La creazione del suddetto sequence diagram, sviluppato a partire dalla descrizione dello scenario del caso d'uso **ConsultazioneSaleDisponibili**, ha evidenziato la necessità di definire alcuni metodi:

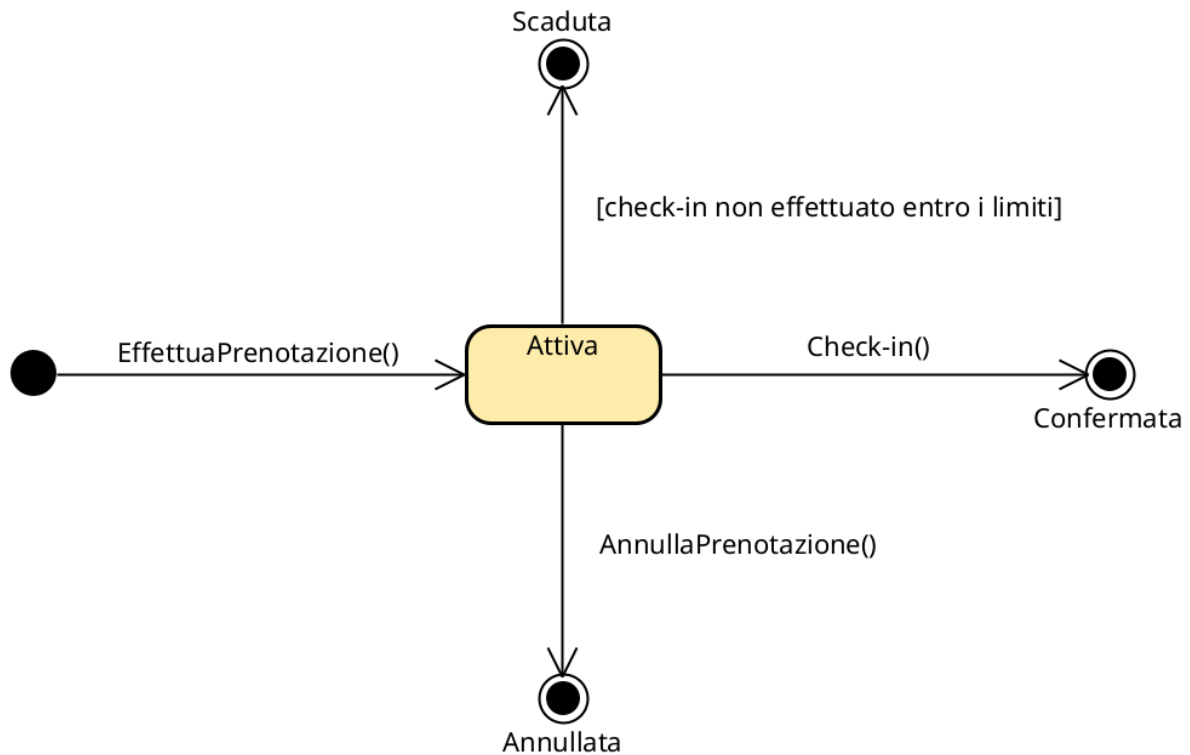
- Nella classe **Biblioteca** il metodo privato `verificaDataInGiorniApertura(data)`, necessario per controllare che la data selezionata dallo Studente rientri nei giorni di apertura previsti per le Sale Studio.
- Nella classe **SalaStudio** il metodo pubblico `getFasceOrariePrestabilite(data)`, necessario per recuperare le fasce orarie predisposte dal Bibliotecario per una determinata Sala Studio nella data selezionata.

Per la verifica della disponibilità delle Postazioni e per il recupero delle Postazioni libere della Sala Studio o delle singole Aree, il sequence diagram riutilizza metodi già individuati nel caso d'uso **EffettuaPrenotazione**; pertanto, tali metodi non vengono riportati nuovamente tra quelli aggiunti.

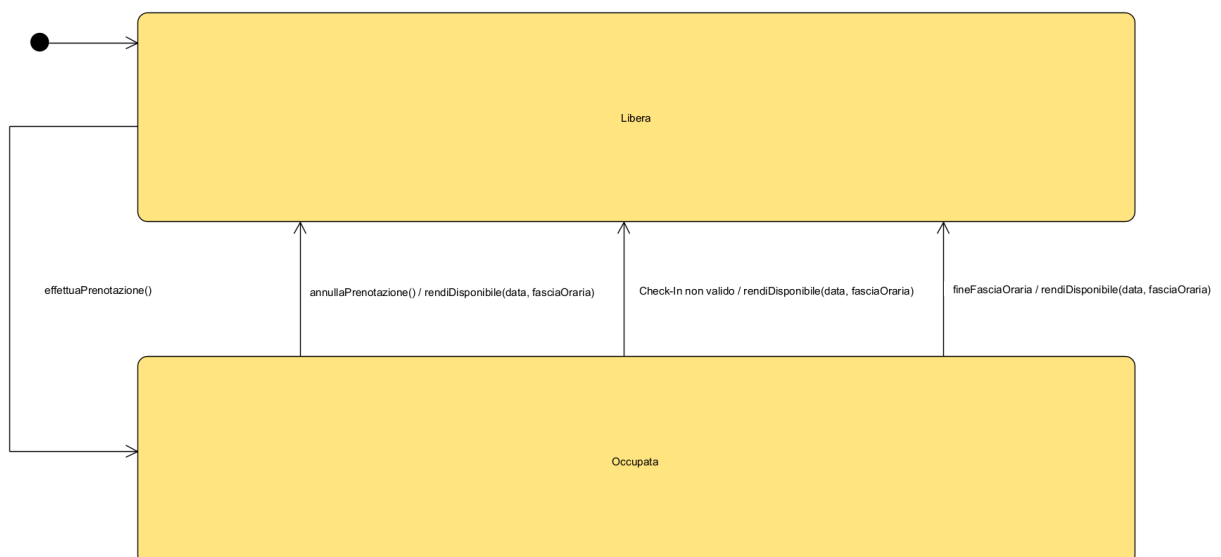


2.8 State Chart Diagram

2.8.1 statoPrenotazione



2.8.2 statoPostazione

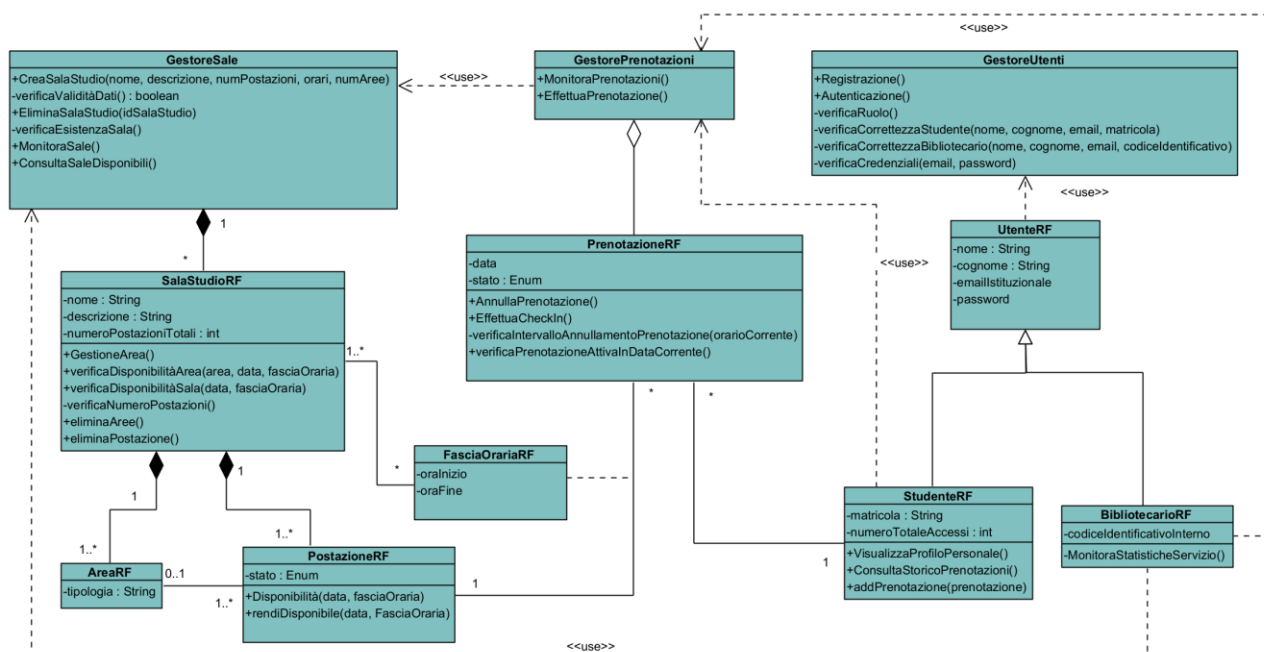


2.9 Tabelle delle responsabilità raffinata

RESPONSABILITÀ	CLASSE
<i>Registrazione</i>	GestoreUtenti
<i>Autenticazione</i>	GestoreUtenti
<i>CreaSalaStudio</i>	GestoreSale
<i>EliminaSalaStudio</i>	GestoreSale
<i>MonitoraPrenotazioni</i>	GestorePrenotazioni
<i>ConsultaSaleDisponibili</i>	GestoreSale
<i>EffettuaPrenotazione</i>	GestorePrenotazioni
<i>VisualizzareProfiloPersonale</i>	Studiante
<i>AnnullaPrenotazione</i>	Prenotazione
<i>EffettuaCheck-in</i>	Prenotazione
<i>MonitoraSale</i>	GestoreSale
<i>MonitoraStatisticheServizio</i>	GestoreReport/GestorePrenotazioni
<i>ConsultaStoricoPrenotazioni</i>	Studiante

2.10 Diagramma delle classi raffinato

Le aggiunte e le modifiche fatte nel corso della costruzione dei Sequence Diagrams hanno determinato lo sviluppo di un **Diagramma delle Classi** raffinato che riporta maggiori dettagli sugli attributi e le principali operazioni delle classi:



3. Piano di test funzionale

Progettare i casi di test funzionale con la tecnica del *Category Partition Testing*. Descrivere il procedimento di calcolo.

Si intende progettare i casi di test funzionale con la tecnica del *Category Partition Testing*.

3.1 Registrazione Utente

REGISTRAZIONE					
RUOLO	NOME	COGNOME	EMAIL ISTITUZIONALI	PASSWORD	MATRICOLA / CODICE
"Studente" oppure "Bibliotecario" - Stringa vuota [ERROR] - Valore non ammesso (es. "Esterno") [ERROR]	- Stringa alfabetica ($1 \leq \text{lung.} \leq 50$) - Stringa vuota [ERROR] - Stringa lung. > 50 [ERROR] - Stringa con numeri/simboli [ERROR]	- Stringa alfabetica ($1 \leq \text{lung.} \leq 50$) - Stringa vuota [ERROR] - Stringa lung. > 50 [ERROR] - Stringa con numeri/simboli [ERROR]	- Formato email valido ($\text{lung.} \leq 255$) - Stringa vuota [ERROR] - Formato non valido [ERROR] - Stringa lung. > 255 [ERROR] - Email già in uso [ERROR]	- Stringa alfanumerica ($8 \leq \text{lung.} \leq 64$) - Stringa vuota [ERROR] - Stringa lung. < 8 [ERROR] - Stringa lung. > 64 [ERROR]	- Formato alfanumerico valido - Stringa vuota [ERROR] - Formato errato [ERROR] - Identificativo già in uso [ERROR]

Il numero di test da effettuarsi senza particolari vincoli è dato dal prodotto di tutte le combinazioni possibili (valide e di errore) per ogni campo:

$$3 \cdot 4 \cdot 4 \cdot 5 \cdot 4 \cdot 4 = 3840$$

Con i vincoli [ERROR], il numero di test da eseguire per testare singolarmente i vincoli isolati è **18** (2 per Ruolo, 3 per Nome, 3 per Cognome, 4 per Email, 3 per Password, 3 per Identificativo). Essendo presenti due flussi di base validi mutuamente esclusivi (uno per lo Studente e uno per il Bibliotecario), calcoliamo 2 test cases a percorso felice (happy path). Il numero di test risultante è:

$$(1 \cdot 1 \cdot 1 \cdot 1 \cdot 1 \cdot 1) \cdot 2 + 18 = 20$$

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Registrazione Studente valida	Tutti input validi (flusso Studente)	Email e matricola inesistenti nel DB.	{Ruolo: "Studente", Nome: "Mario", Cognome: "Rossi", Email: "m.rossi@studenti.unina.it", Password: "Password123!", Matricola: "N86001234"}	Conferma registrazione	Profilo Studente creato.
2	Registrazione Bibliotecario valida	Tutti input validi (flusso Bibliotecario)	Email e codice inesistenti nel DB.	{Ruolo: "Bibliotecario", Nome: "Laura", Cognome: "Bianchi", Email: "l.bianchi@unina.it",	Conferma registrazione.	Profilo Bibliotecario creato.



				Password: "SecurePass8!", Codice: "BIB01"}		
3	Ruolo vuoto	Ruolo vuoto [ERROR], altri validi		{Ruolo: "", Nome: "Mario", ...}	"Il ruolo è obbligatorio."	Nessun utente creato.
4	Ruolo non ammesso	Ruolo non ammesso [ERROR], altri validi		{Ruolo: "Docente", Nome: "Mario", ...}	"Ruolo selezionato non valido."	Nessun utente creato.
5	Nome vuoto	Nome vuoto [ERROR], altri validi		{Nome: "", Cognome: "Rossi", ...}	"Il nome è obbligatorio."	Nessun utente creato.
6	Nome > 50 caratteri	Nome > 50 chars [ERROR], altri validi		{Nome: "[Stringa di 51 caratteri]", ...}	"Nome troppo lungo (max 50 caratteri)."	Nessun utente creato.
7	Nome con caratteri speciali	Nome, Cognome validi, Patente stringa < 10 caratteri [ERROR], TipoPatente, Credito validi		{Nome: "M4rio!", Cognome: "Rossi", ...}	"Formato nome non valido."	Nessun utente creato.
8	Cognome vuoto	Cognome vuoto [ERROR], altri validi		{Nome: "Mario", Cognome: "", ...}	"Il cognome è obbligatorio."	Nessun utente creato.
9	Cognome > 50 caratteri	Cognome > 50 chars [ERROR], altri validi		{Cognome: "[Stringa di 51 caratteri]", ...}	"Cognome troppo lungo (max 50 caratteri)."	Nessun utente creato.
10	Cognome con caratteri speciali	Cognome con simboli [ERROR], altri validi		{Cognome: "Rossi_86", ...}	"Formato cognome non valido."	Nessun utente creato.
11	Email vuota	Email vuota [ERROR], altri validi		{Email: "", ...}	"L'email è obbligatoria."	Nessun utente creato.
12	Formato email errato	Email errata [ERROR], altri validi		{Email: "m.rossistudenti.unina.it", ...}	"Formato email non valido."	Nessun utente creato.
13	Email > 255 caratteri	Email > 255 chars [ERROR], altri validi		{Email: "[Stringa > 255 char]@unina.it", ...}	"Email troppo lunga (max 255 caratteri)."	Nessun utente creato.
14	Email già in uso	Email già presente [ERROR], altri validi	Email già associata a un utente.	{Email: "m.rossi@studenti.unina.it", ...}	"Email già associata a un account."	Nessun utente creato.
15	Password vuota	Password vuota [ERROR], altri validi		{Password: "", ...}	"La password è obbligatoria."	Nessun utente creato.
16	Password < 8 caratteri	Password < 8 chars [ERROR], altri validi		{Password: "Pass1!", ...}	"La password deve contenere almeno 8 caratteri."	Nessun utente creato.
17	Password > 64 caratteri	Password > 64 chars [ERROR], altri validi		{Password: "[Stringa di 65 caratteri]", ...}	"Password troppo lunga (max 64 caratteri)."	Nessun utente creato.
18	Identificativo vuoto	Matricola/Codice vuoto [ERROR], altri validi		{Matricola: "", ...}	"Il codice identificativo è obbligatorio."	Nessun utente creato.



19	Identificativo formato errato	Matricola/Codice errato [ERROR], altri validi		{Matricola: "M86001234", ...}	"Formato identificativo non riconosciuto."	Nessun utente creato.
20	Identificativo già in uso	Matricola/Codice presente [ERROR], altri validi	Identificativo già associato a un utente.	{Matricola: "N86001234", ...}	"Identificativo già in uso nel sistema."	Nessun utente creato.

3.2 Autenticazione Utente

Autenticazione	
EMAIL ISTITUZIONALE	PASSWORD
▪ Formato email valido (lunghezza ≤ 255) e registrata a sistema ▪ Email non presente nel sistema [ERROR] ▪ Stringa vuota [ERROR] ▪ Formato non valido (es. mancante di @) [ERROR] ▪ Stringa lunghezza > 255 [ERROR]	▪ Stringa alfanumerica ($8 \leq \text{lunghezza} \leq 64$) e corrispondente all'email ▪ Password errata (non corrispondente all'account) [ERROR] ▪ Stringa vuota [ERROR] ▪ Stringa lunghezza < 8 [ERROR] ▪ Stringa lunghezza > 64 [ERROR]

Il numero di test da effettuarsi senza particolari vincoli è dato dal prodotto di tutte le combinazioni possibili (valide e di errore) per ogni campo di input:

$$5 \times 5 = 25$$

Con i vincoli [ERROR], il numero di test da eseguire per testare singolarmente le condizioni di errore isolate è **8** (4 per l'Email Istituzionale, 4 per la Password).

Volendo testare i due flussi di base validi per le due tipologie di utente previste dal sistema (Studente e Bibliotecario), calcoliamo 2 test cases a percorso felice (happy path). Il numero di test risultante è:

$$(1 \times 1) + 8 = 9$$

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Autenticazione valida	Tutti input validi	L'utente è regolarmente registrato nel sistema.	{Email: "m.rossi@studenti.unina.it", Password: "Password123!"}	Accesso consentito, reindirizzamento alla Dashboard Studente.	L'utente risulta autenticato nel sistema.
2	Email inesistente	Email non presente [ERROR], password formato valido	Nessun utente è registrato con l'email fornita.	{Email: "utente.fantasma@unina.it", Password: "Password123!"}	"Credenziali non valide o utente inesistente."	Nessun utente autenticato.
3	Email vuota	Email vuota [ERROR], password formato valido		{Email: "", Password: "Password123!"}	"L'email è obbligatoria."	Nessun utente autenticato.
4	Formato email errato	Formato errato [ERROR], password formato valido		{Email: "m.rossistudenti.unina.it", Password: "Password123!"}	"Formato email non valido."	Nessun utente autenticato.

5	Email > 255 caratteri	Email > 255 chars [ERROR], password formato valido		{Email: "[Stringa > 255 char]@unina.it", Password: "Password123!"}	"Formato email non valido."	Nessun utente autenticato.
6	Password errata	Password non corrispondente [ERROR], email valida	L'utente è registrato ma la password fornita non è quella corretta.	{Email: "m.rossi@studenti.unina.it", Password: "PasswordErrata99!"}	"Credenziali non valide."	Nessun utente autenticato.
7	Password vuota	Password vuota [ERROR], email valida	L'utente è registrato nel sistema.	{Email: "m.rossi@studenti.unina.it", Password: ""}	"La password è obbligatoria."	Nessun utente autenticato.
8	Password < 8 caratteri	Password < 8 chars [ERROR], email valida	L'utente è registrato nel sistema.	{Email: "m.rossi@studenti.unina.it", Password: "Pass1"}	"La password deve contenere almeno 8 caratteri."	Nessun utente autenticato.
9	Password > 64 caratteri	Password > 64 chars [ERROR], email valida	L'utente è registrato nel sistema.	{Email: "m.rossi@studenti.unina.it", Password: "[Stringa di 65 caratteri]"}	"Formato password non valido."	Nessun utente autenticato.

3.3 Effettua Prenotazione

EffettuaPrenotazione				
SALA STUDIO	DATA	FASCIA ORARIA	AREA	POSTAZIONE
- Id di sala esistente nel sistema - Nessuna Sala selezionata [ERROR] - Sala con identificativo non esistente [ERROR] - Sala selezionata non contenente alcuna postazione disponibile, per fascia oraria selezionata [ERROR] - Sala selezionata non contenente alcuna postazione disponibile, per data selezionata [ERROR]	- Data nel formato GG/MM/AA, futura o odierna, in giorno di apertura - Data nel passato [ERROR] - Formato data non valido (es. 32/13/20226) [ERROR] - Data in giorno di chiusura della biblioteca [ERROR]	- Formato HH:MM-HH-MM con inizio < fine e facente parte delle fasce scelte dal bibliotecario - Formato fascia oraria non valido (es. lettere, 25:00, fine > inizio) [ERROR] - Fascia oraria non compresa tra quelle scelte dal bibliotecario per la determinata sala [ERROR]	- Area specifica esistente nella sala selezionata - Nessuna area selezionata (assegnazione sull'intera sala) - Area non esistente nella sala selezionata [ERROR] - Area selezionata non contenente alcuna postazione libera nella data selezionata [ERROR] - Area selezionata non contenente alcuna postazione libera nella fascia oraria	- Postazione specifica disponibile selezionata - Richiesta assegnazione automatica (postazione non selezionata) - Postazione specifica selezionata non disponibile [ERROR] - Postazione assegnata alla prenotazione (che sia scelta dallo studente o in modo automatico) non più disponibile all'atto della conferma [ERROR]

			selezionata [ERROR]	
--	--	--	------------------------	--

Il numero di test da effettuarsi senza particolari vincoli è: $5 \cdot 4 \cdot 3 \cdot 5 \cdot 4 = 1200$.

Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è 14 (4 per Sala Studio, 3 per data, 2 per fascia oraria, 3 per Area, 2 per Postazione).

Il numero di test risultante è: $(1 \cdot 1 \cdot 1 \cdot 2 \cdot 2) + 14 = 18$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti validi — area specificata, postazione specifica	Sala valida, Data valida, Fascia valida, Area specificata valida, Postazione specifica disponibile		{Sala: "Sala Caccioppoli", Data: "30/06/2026", FasciaOraria: "09:30-10:30", Area: "Area Silenzio", Postazione: "P15"}	Conferma prenotazione	Prenotazione in stato "attiva"; postazione P15 della Sala Caccioppoli occupata per 30/06/2026 09:00-13:00; visibile nel profilo dello Studente
2	Tutti validi — area specificata, assegnazione automatica	Sala valida, Data valida, Fascia valida, Area specificata valida, Assegnazione automatica		{Sala: "Sala Pancini", Data: "01/07/2026", FasciaOraria: "14:30-15:30", Area: "Area Gruppo Studio", Postazione: null}	Conferma prenotazione con indicazione della postazione assegnata dal sistema (prima libera nell'area)	Prenotazione in stato "attiva" con postazione assegnata nell'Area Gruppo Studio della Sala Pancini
3	Tutti validi — nessuna area, postazione specifica	Sala valida, Data valida, Fascia valida, Nessuna area, Postazione specifica disponibile		{Sala: "Sala DIETI", Data: "02/07/2026", FasciaOraria: "09:30-10:30", Area: null, Postazione: "P03"}	Conferma prenotazione	Prenotazione in stato "attiva"; postazione P03 della Sala DIETI (non associata a un'area) occupata per 02/07/2026 09:00-13:00

4	Tutti validi — nessuna area, assegnazione automatica	Sala valida, Data valida, Fascia valida, Nessuna area, Assegnazione automatica		{Sala: "Sala Filangieri", Data: "03/07/2026", FasciaOraria: "14:30-15:30", Area: null, Postazione: null}	Conferma prenotazione con postazione assegnata automaticamente all'interno della sala	Prenotazione in stato "attiva" con postazione assegnata dal sistema nella Sala selezionata
5	Nessuna sala selezionata	Sala non selezionata [ERROR] , Data valida, Fascia valida, Area valida, Postazione valida		{Sala: null, Data: "30/06/2026", FasciaOraria: "09:30-10:30", Area: "Area Silenzio", Postazione: "P15"}	La selezione della sala studio è obbligatoria	Nessuna prenotazione creata
6	Sala con identificativo non esistente	Sala non esistente [ERROR] , Data valida, Fascia valida, Area valida, Postazione valida		{Sala: "Sala Fantasma", Data: "30/06/2026", FasciaOraria: "09:30-10:30", Area: "Area Silenzio", Postazione: "P15"}	Sala studio non trovata. Selezionare una sala esistente	Nessuna prenotazione creata
7	Sala saturata per la fascia oraria selezionata	Sala senza postazioni libere per la fascia [ERROR] , Data valida, Fascia valida, Area valida, Postazione valida		{Sala: "Sala Volta", Data: "04/07/2026", FasciaOraria: "10:30-11:30", Area: null, Postazione: null}	Nessuna postazione disponibile nella sala studio per la fascia oraria richiesta. Il caso d'uso termina	Nessuna prenotazione creata
8	Sala saturata per la data selezionata (tutte le fasce)	Sala senza postazioni libere per la data [ERROR] , Data valida, Fascia valida, Area valida, Postazione valida		{Sala: "Sala Volta", Data: "05/07/2026", FasciaOraria: "09:30-10:30", Area: null, Postazione: null}	Nessuna postazione disponibile nella sala studio per la data richiesta. Il caso d'uso termina	Nessuna prenotazione creata
9	Data nel passato	Sala valida, Data nel passato [ERROR] , Fascia valida, Area valida, Postazione valida		{Sala: "Sala Caccioppoli", Data: "10/01/2024", FasciaOraria: "09:30-10:30", Area: "Area Silenzio", Postazione: "P15"}	Non è possibile prenotare una data nel passato	Nessuna prenotazione creata
10	Formato data non valido	Sala valida, Formato data non valido [ERROR] , Fascia valida, Area valida, Postazione valida		{Sala: "Sala Caccioppoli", Data: "32/13/2026", FasciaOraria: "09:30-10:30", Area: "Area Silenzio", Postazione: "P15"}	Formato data non valido. Utilizzare il formato GG/MM/AAAA	Nessuna prenotazione creata
11	Data in giorno di chiusura della biblioteca	Sala valida, Data in giorno di chiusura [ERROR] , Fascia valida, Area valida,		{Sala: "Sala Caccioppoli", Data: "21/06/2026", FasciaOraria: "09:30-10:30", Area: "Area Silenzio", Postazione: "P15"}	La biblioteca è chiusa nel giorno selezionato. Selezionare un'altra data	Nessuna prenotazione creata

		Postazione valida				
12	Formato fascia oraria non valido	Sala valida, Data valida, Formato fascia oraria non valido [ERROR] , Area valida, Postazione valida		{Sala: "Sala Caccioppoli", Data: "30/06/2026", FasciaOraria: "25:00-30:00", Area: "Area Silenzio", Postazione: "P15"}	Formato fascia oraria non valido. Utilizzare HH:MM-HH:MM con inizio < fine	Nessuna prenotazione creata
13	Fascia non prevista dal bibliotecario per la sala	Sala valida, Data valida, Fascia non tra quelle previste per la sala [ERROR] , Area valida, Postazione valida		{Sala: "Sala Caccioppoli", Data: "30/06/2026", FasciaOraria: "06:00-07:00", Area: "Area Silenzio", Postazione: "P15"}	La fascia oraria selezionata non è tra quelle previste per la sala	Nessuna prenotazione creata
14	Area non esistente nella sala selezionata	Sala valida, Data valida, Fascia valida, Area non esistente nella sala [ERROR] , Postazione valida		{Sala: "Sala Caccioppoli", Data: "30/06/2026", FasciaOraria: "09:30-10:30", Area: "Area Inesistente", Postazione: null}	Area non trovata nella sala studio selezionata	Nessuna prenotazione creata
15	Area satura per la data selezionata (tutte le fasce)	Sala valida, Data valida, Fascia valida, Area senza postazioni libere per la data [ERROR] , Postazione valida		{Sala: "Sala Caccioppoli", Data: "06/07/2026", FasciaOraria: "09:30-10:30", Area: "Area Computer", Postazione: null}	Nessuna postazione disponibile nell'area selezionata per la data richiesta. Il caso d'uso termina	Nessuna prenotazione creata
16	Area satura per la fascia oraria selezionata	Sala valida, Data valida, Fascia valida, Area senza postazioni libere per la fascia [ERROR] , Postazione valida		{Sala: "Sala Caccioppoli", Data: "30/06/2026", FasciaOraria: "10:30-11:30", Area: "Area Computer", Postazione: null}	Nessuna postazione disponibile nell'area selezionata per la fascia oraria richiesta. Il caso d'uso termina	Nessuna prenotazione creata
17	Postazione specifica selezionata non disponibile al momento della selezione	Sala valida, Data valida, Fascia valida, Area valida, Postazione specifica non disponibile [ERROR]		{Sala: "Sala Pancini", Data: "30/06/2026", FasciaOraria: "09:30-10:30", Area: "Area Silenzio", Postazione: "P22"}	La postazione selezionata non è disponibile per la fascia oraria richiesta. Selezionare un'altra postazione o richiedere l'assegnazione automatica	Nessuna prenotazione creata
18	Postazione assegnata non più disponibile all'atto della conferma (race)	Sala valida, Data valida, Fascia valida, Area valida, Postazione non più disponibile		{Sala: "Sala Pancini", Data: "30/06/2026", FasciaOraria: "09:00-13:00", Area: "Area Silenzio", Postazione: "P15"}	La postazione P15 è stata appena prenotata da un altro Studente.	Nessuna prenotazione creata. Il sistema rappresenta le postazioni

	condition, scenario 8.1)	alla conferma [ERROR]			Selezionare una postazione diversa o richiedere l'assegnazion e automatica	disponibili (ritorno al punto 7 dello scenario)
--	-----------------------------	----------------------------------	--	--	--	---

3.4 Creazione Sala Studio

CreaSalaStudio				
NOME SALA	DESCRIZIONE	NUMERO POSTAZIONI	ORARI	NUMERO AREE
- Stringa alfanumerica ($1 \leq \text{lunghezza} \leq 50$) - Stringa vuota [ERROR] - Stringa di caratteri di lunghezza > 50 [ERROR] - Stringa che contiene caratteri speciali non ammessi [ERROR]	- Stringa alfanumerica ($\text{lunghezza} \leq 256$) - Stringa alfanumerica con lunghezza > 256 [ERROR] - Stringa che contiene caratteri speciali non ammessi [ERROR]	- Numero intero positivo (> 0) - Numero intero ≤ 0 [ERROR] - Caratteri non numerici [ERROR] - Tipo di dato numerico non ammesso (float, double) [ERROR]	- Formato orario valido (HH:MM) con Apertura $<$ Chiusura - Formato orario non valido (es. lettere o 25:00) [ERROR] - Orario Apertura $\geq$ Orario Chiusura [ERROR]	- Numero intero positivo (≥ 1) - Numero intero ≤ 0 [ERROR] - Caratteri non numerici [ERROR] - Tipo di dato numerico non ammesso (float, double) [ERROR]

Il numero di test da effettuarsi senza particolari vincoli è: $4 * 3 * 4 * 3 * 4 = 576$.

Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è 13 (3 per Nome Sala, 2 per Descrizione, 3 per Numero Postazioni, 2 per Orario, 3 per Numero Aree). Il numero di test risultante è: $(1 * 1 * 1 * 1 * 1) + 13 = 14$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi	Nome valido, Descrizione valida, Postazioni valide, Orario valido, Aree valide		{Nome: "Sala Newton", Descrizione: "Sala silenziosa piano terra", NumPostazioni: "50", Orario: "08:30 - 18:30", NumAree: "2"}	Messaggio di conferma creazione.	La nuova Sala Studio è memorizzata nel sistema.
2	Nome vuoto	Nome stringa vuota [ERROR], tutti gli altri validi		{Nome: "", Descrizione: "Sala silenziosa piano terra", NumPostazioni: "50", Orario: "08:30 - 18:30", NumAree: "2"}	Il nome della sala è obbligatorio.	Nessuna sala creata.
3	Nome stringa > 50 caratteri	Nome stringa > 50 [ERROR], tutti gli altri validi		{Nome: "Sala Newton dedicata agli studenti di Ingegneria Informatica", Descrizione: "Sala silenziosa", NumPostazioni: "50", Orario: "08:30 - 18:30", NumAree: "2"}	Nome troppo lungo (max 50 caratteri).	Nessuna sala creata.
4	Nome con caratteri speciali	Nome con caratteri speciali non ammessi [ERROR], tutti gli altri validi		{Nome: "Sala Newton @#", Descrizione: "Sala silenziosa", NumPostazioni: "50", Orario: "08:30 - 18:30", NumAree: "2"}	Formato nome non valido.	Nessuna sala creata.
5	Descrizione > 256 caratteri	Descrizione > 256 [ERROR], tutti gli altri validi		{Nome: "Sala Newton", Descrizione: "[Stringa di 256 caratteri...]", NumPostazioni: "50", Orario: "08:30 - 18:30", NumAree: "2"}	Descrizione troppo lunga.	Nessuna sala creata.
6	Descrizione con caratteri speciali	Descrizione contenente caratteri speciali		{Nome: "Sala Newton", Descrizione: "Sala silenziosa @#", NumPostazioni: "50",	Caratteri speciali non ammessi	Nessuna sala creata.

		[ERROR], tutti gli altri validi		Orario: "08:30-18:30", NumAree: "2"}	nella descrizione.	
7	Postazioni <= 0	Postazioni ≤ 0 [ERROR], tutti gli altri validi		{Nome: "Sala Newton", Descrizione: "Sala silenziosa", NumPostazioni: "0", Orario: "08:30 - 18:30", NumAree: "2"}	La sala deve avere almeno una postazione.	Nessuna sala creata.
8	Postazioni non numeriche	Postazioni non numeriche [ERROR], tutti gli altri validi		{Nome: "Sala Newton", Descrizione: "Sala silenziosa", NumPostazioni: "Cinquanta", Orario: "08:30 - 18:30", NumAree: "2"}	Inserire un valore numerico valido.	Nessuna sala creata.
9	Postazioni valore decimale (float/double)	Tipo di dato numerico non ammesso per Postazioni [ERROR], tutti gli altri validi		{Nome: "Sala Newton", Descrizione: "Sala silenziosa", NumPostazioni: "50.5", Orario: "08:30-18:30", NumAree: "2"}	Il numero di postazioni deve essere un intero.	Nessuna sala creata.
10	Orario formato non valido	Orario formato errato [ERROR], tutti gli altri validi		{Nome: "Sala Newton", Descrizione: "Sala silenziosa", NumPostazioni: "50", Orario: "25:00 - 18:30", NumAree: "2"}	Formato orario non valido.	Nessuna sala creata.
11	Apertura ≥ Chiusura	Apertura ≤ Chiusura [ERROR], tutti gli altri validi		{Nome: "Sala Newton", Descrizione: "Sala silenziosa", NumPostazioni: "50", Orario: "18:30 - 08:30", NumAree: "2"}	L'orario di chiusura deve essere successivo all'apertura.	Nessuna sala creata.
12	Numero Aree ≤ 0	Aree ≤ 0 [ERROR], tutti gli altri validi		{Nome: "Sala Newton", Descrizione: "Sala silenziosa", NumPostazioni: "50", Orario: "08:30 - 18:30", NumAree: "0"}	La sala deve avere almeno un'area.	Nessuna sala creata.
13	Numero Aree non numerico	Aree non numeriche [ERROR], tutti gli altri validi		{Nome: "Sala Newton", Descrizione: "Sala silenziosa", NumPostazioni: "50", Orario: "08:30 - 18:30", NumAree: "Due"}	Inserire un valore numerico valido per le aree.	Nessuna sala creata.
14	Aree valore decimale (float/double)	Tipo di dato numerico non ammesso per Aree [ERROR], tutti gli altri validi		{Nome: "Sala Newton", Descrizione: "Sala silenziosa", NumPostazioni: "50", Orario: "08:30-18:30", NumAree: "2.5"}	Il numero di aree deve essere un intero.	Nessuna sala creata.

3.5 Eliminazione Sala Studio

ID SALA	MOTIVAZIONE
- Numero intero positivo (> 0) associato a una sala esistente - Numero zero o negativo (≤ 0) [ERROR] - Caratteri non numerici [ERROR] - ID numerico valido ma non esistente nel sistema [ERROR]	- Stringa alfanumerica (lunghezza ≤ 150) - Stringa troppo lunga (> 150 caratteri) [ERROR]

Il numero di test da effettuarsi senza particolari vincoli è: $4 * 2 = 8$. Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è 4 (3 per ID Sala, 1 per Motivazione). Il numero di test risultante è: $(1 * 1) + 4 = 5$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1	Tutti input validi	ID Sala valido ed esistente, Motivazione valida	La sala con ID 12 esiste nel sistema.	{IDSala: "12", Motivazione: "Chiusura per lavori di ristrutturazione"}	Messaggio di conferma eliminazione.	Si ricevono le credenziali di accesso per email
2	ID Sala zero o negativo	ID Sala ≤ 0 [ERROR], motivazione valida		{IDSala: "0", Motivazione: "Manutenzione"}	ID Sala non valido.	Nessuna sala viene eliminata.
3	ID Sala non numerico	ID Sala non numerico [ERROR], motivazione valida		{IDSala: "dodici", Motivazione: "Manutenzione"}	L'ID della sala deve essere un valore numerico.	Nessuna sala viene eliminata.
4	ID Sala inesistente	ID Sala non esistente [ERROR], motivazione valida	La sala con ID 999 non esiste nel sistema.	{IDSala: "999", Motivazione: "Manutenzione"}	La sala selezionata non esiste.	Nessuna sala viene eliminata.
5	Motivazione troppo lunga	Motivazione > 150 caratteri [ERROR], ID Sala valido	La sala con ID 12 esiste nel sistema.	{IDSala: "12", Motivazione: "[Stringa di 151 caratteri...]"}}	La motivazione supera la lunghezza massima consentita.	Nessuna sala viene eliminata.

3.6 Annullamento Prenotazione

Check-in Postazione				
PLACEHOLDER	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER
▪	▪	▪	▪	▪

Il numero di test da effettuarsi senza particolari vincoli è: $c1 * c2 * c3 * c4 * c5 = n$.

Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è k (k1 per [PLACEHOLDER], k2 per [PLACEHOLDER], k3 per [PLACEHOLDER], 2 per [PLACEHOLDER], 2 per [PLACEHOLDER]).

Il numero di test risultante è: $(c1b*c2b*c3b*c4b*c5b) + k = w$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1						

3.7 Check-in Postazione

Check-in Postazione				
PLACEHOLDER	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER
▪	▪	▪	▪	▪

Il numero di test da effettuarsi senza particolari vincoli è: $c1 * c2 * c3 * c4 * c5 = n$.

Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è k (k1 per [PLACEHOLDER], k2 per [PLACEHOLDER], k3 per [PLACEHOLDER], 2 per [PLACEHOLDER], 2 per [PLACEHOLDER]).

Il numero di test risultante è: $(c1b*c2b*c3b*c4b*c5b) + k = w$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1						

3.8 VisualizzaProfiloPersonale

Check-in Postazione				
PLACEHOLDER	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER
▪	▪	▪	▪	▪

Il numero di test da effettuarsi senza particolari vincoli è: $c1 * c2 * c3 * c4 * c5 = n$.

Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è k (k1 per [PLACEHOLDER], k2 per [PLACEHOLDER], k3 per [PLACEHOLDER], 2 per [PLACEHOLDER], 2 per [PLACEHOLDER]).

Il numero di test risultante è: $(c1b*c2b*c3b*c4b*c5b) + k = w$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1						

3.9 ConsultazioneSaleDisponibili

Check-in Postazione				
PLACEHOLDER	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER	PLACEHOLDER
▪	▪	▪	▪	▪

Il numero di test da effettuarsi senza particolari vincoli è: $c1 * c2 * c3 * c4 * c5 = n$.

Con i vincoli [ERROR], invece, il numero di test da eseguire per testare singolarmente i vincoli è k (k1 per [PLACEHOLDER], k2 per [PLACEHOLDER], k3 per [PLACEHOLDER], 2 per [PLACEHOLDER], 2 per [PLACEHOLDER]).

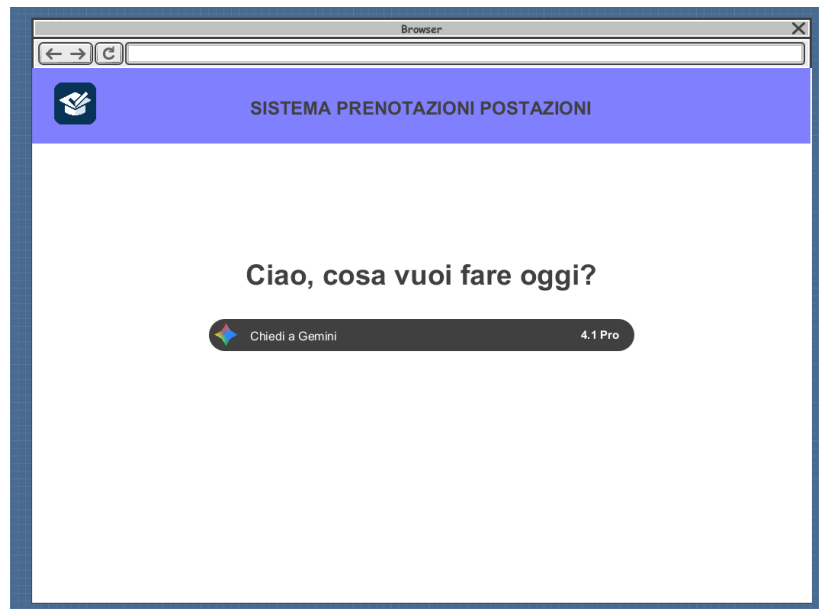
Il numero di test risultante è: $(c1b*c2b*c3b*c4b*c5b) + k = w$.

TEST SUITE						
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese
1						

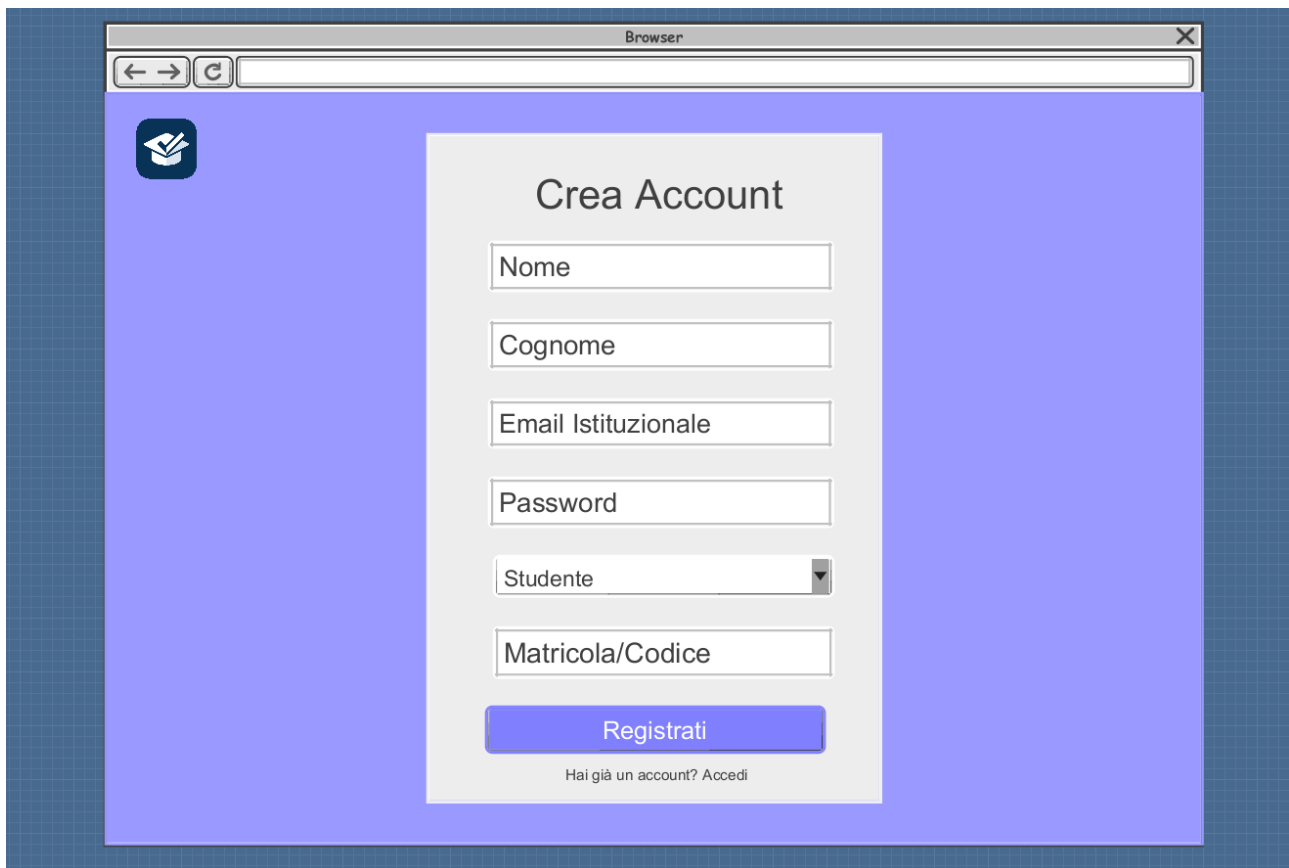
4. Progettazione

4.1 Wireframes

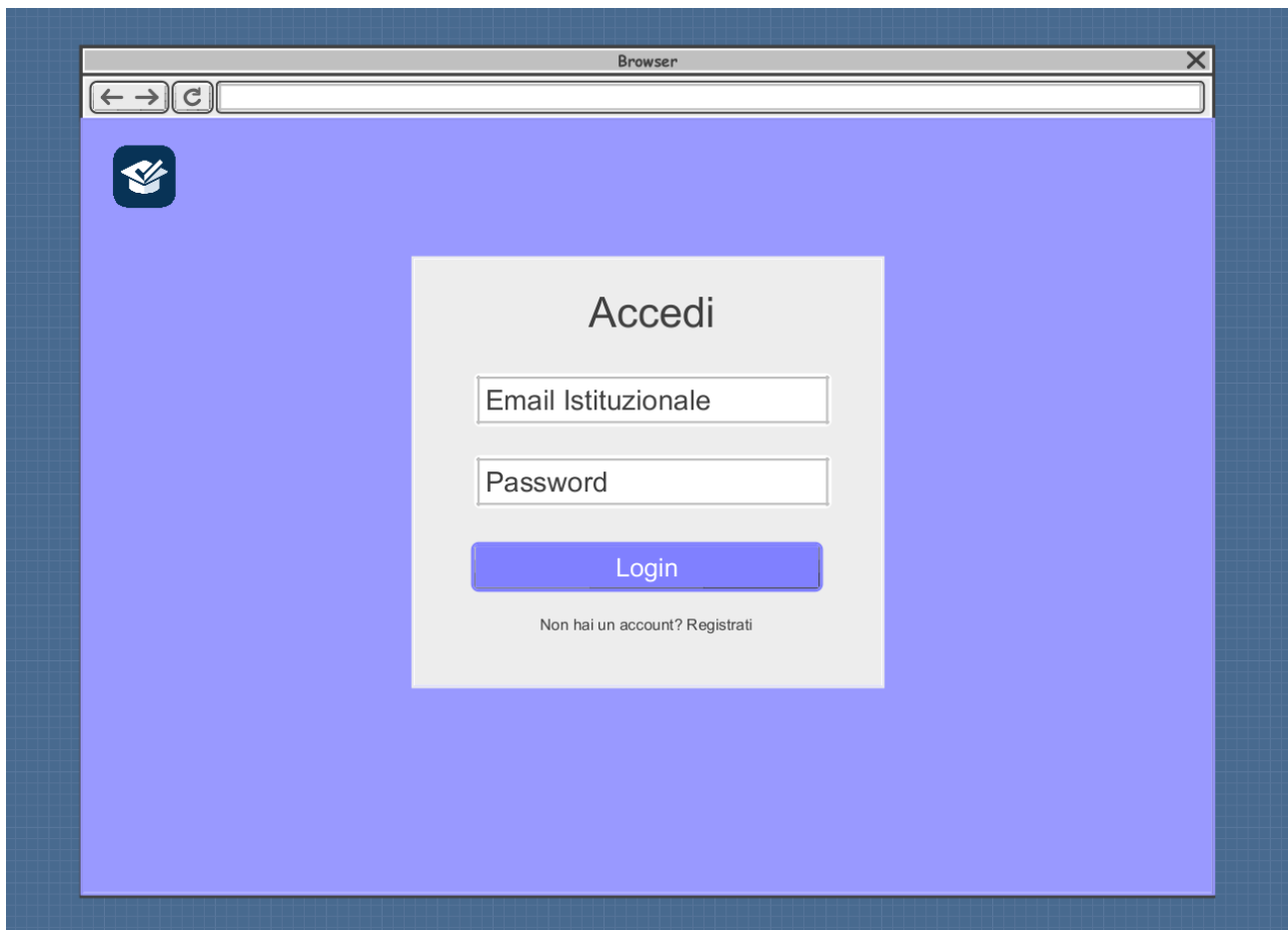
4.1.1 Homepage



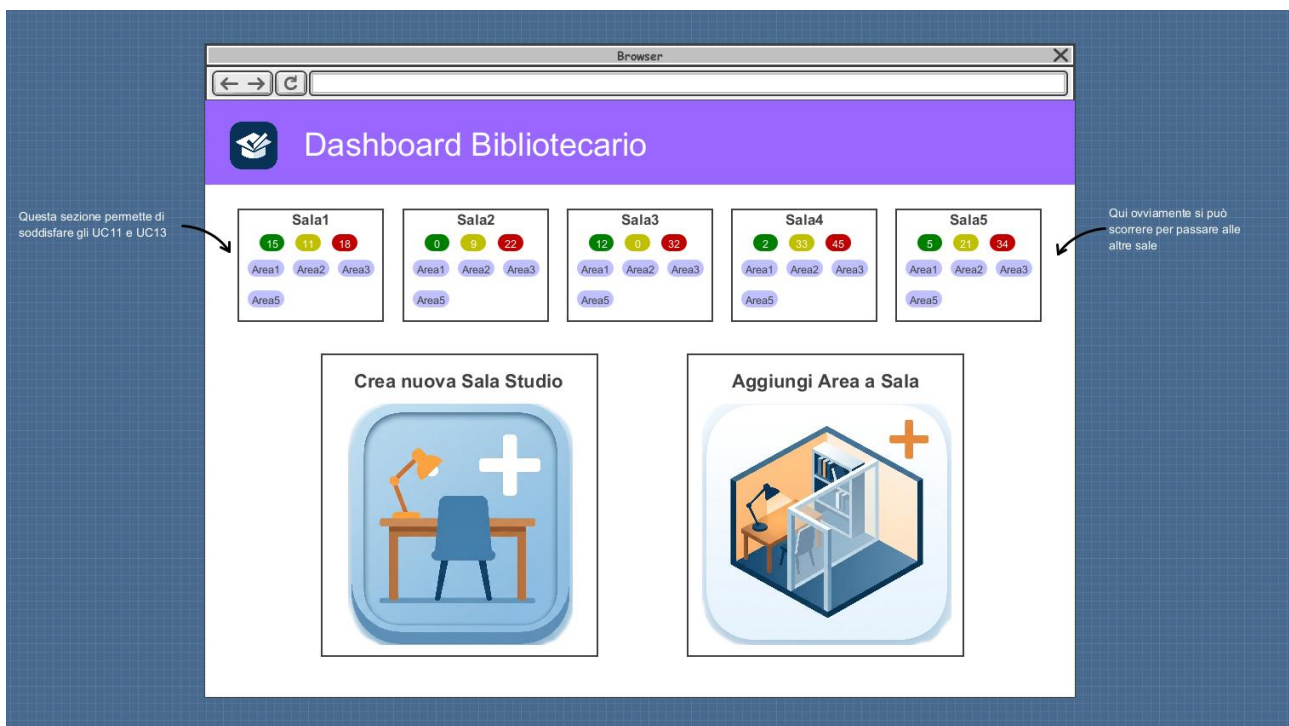
4.1.2 Registrazione



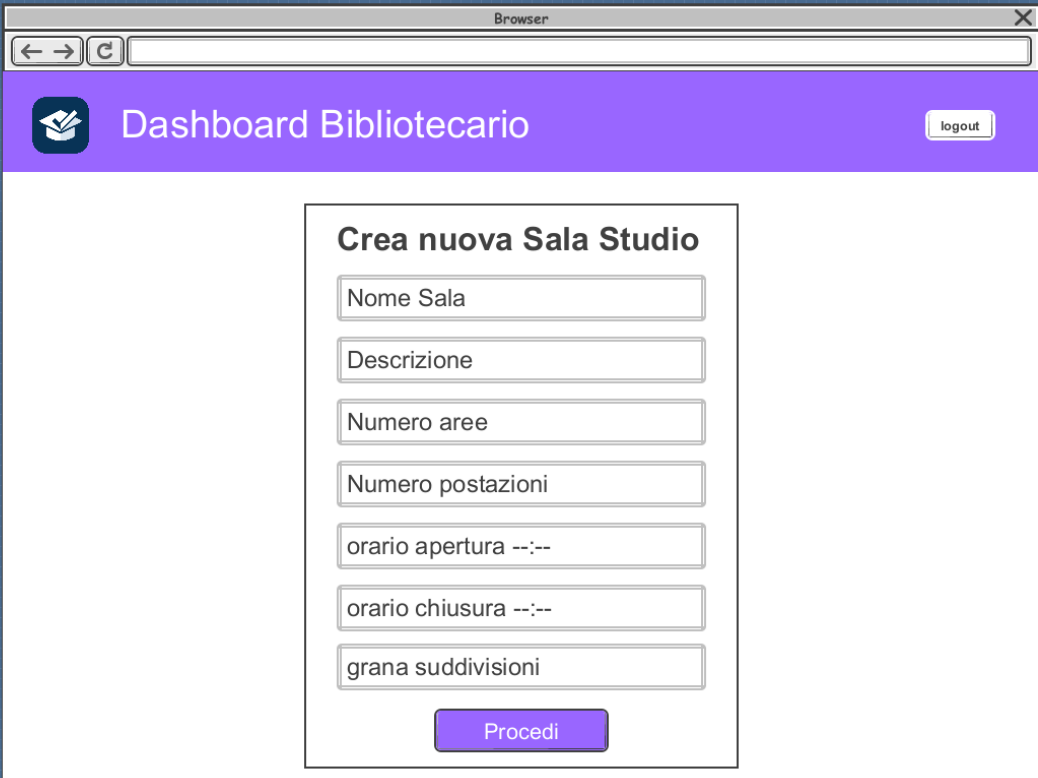
4.1.3 Accesso



4.1.4 Dashboard Bibliotecario

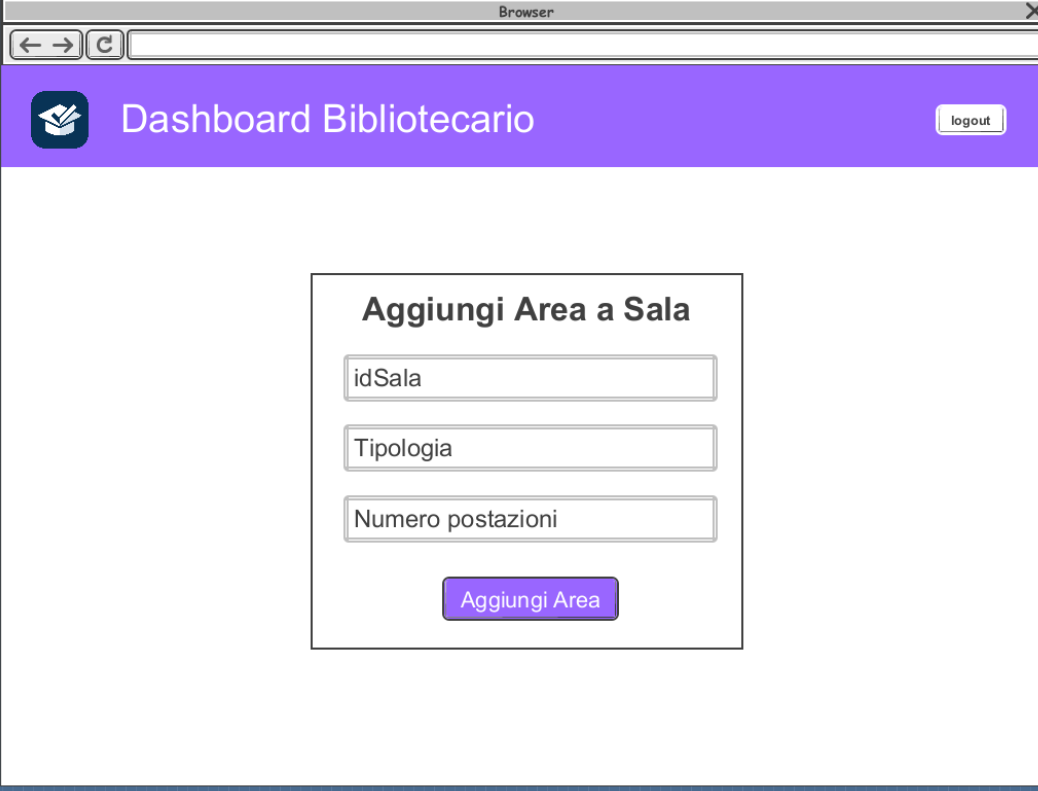


4.1.5 Crea nuova Sala Studio



The screenshot shows a web browser window with the title 'Browser'. The page has a purple header bar with a book icon, the text 'Dashboard Bibliotecario', and a 'logout' button. The main content area is white and contains a form titled 'Crea nuova Sala Studio'. The form has several input fields: 'Nome Sala', 'Descrizione', 'Numero aree', 'Numero postazioni', 'orario apertura --:--', 'orario chiusura --:--', and 'grana suddivisioni'. At the bottom of the form is a purple button labeled 'Procedi'.

4.1.6 Aggiungi Area a Sala



The screenshot shows a web browser window with the title 'Browser'. The page has a purple header bar with a book icon, the text 'Dashboard Bibliotecario', and a 'logout' button. The main content area is white and contains a form titled 'Aggiungi Area a Sala'. The form has three input fields: 'idSala', 'Tipologia', and 'Numero postazioni'. At the bottom of the form is a purple button labeled 'Aggiungi Area'.

4.1.7 Dashboard Studente

4.1.8 Profilo Personale

Browser

Dashboard Studente

logout

Profilo Personale di \$matricola

Totale accessi: 4

Prenotazioni

ordina per: data

opzionale, si vedrà in progettazione

Qui con l'aumentare delle prenotazioni continuerà a riempirsi la pagina

Prenotazione #1 **ATTIVA**
Data: 16-06-2026
Orario: 8:30-10:30
Sala1 - Area1 - Postazione #1
Effettua Check-in Annulla

Prenotazione #2 **CONFERMATA**
Data: 17-06-2026
Orario: 8:30-10:30
Sala1 - Area1 - Postazione #1

Prenotazione #3 **SCADUTA**
Data: 13-06-2026
Orario: 8:30-10:30
Sala1 - Area1 - Postazione #1

Prenotazione #4 **ANNULLATA**
Data: 19-06-2026
Orario: 8:30-10:30
Sala1 - Area1 - Postazione #1

4.1.9 Effettua Prenotazione

Browser

Dashboard Studente

logout

1 Sala & Data 2 Fascia Oraria 3 Area 4 Postazione 4 Conferma

1. Scegli la Sala

È scrollabile

☐ Piazzale Tecchio I
50/80 postazioni disponibili totali

☒ Piazzale Tecchio II
30/40 postazioni disponibili totali

☐ Via Claudio I
22/90 postazioni disponibili totali

☐ Via Claudio II
10/40 postazioni totali

☐ San Giovanni I
0/60 postazioni totali

2. Scegli la data

Giugno

<< >>

L	M	M	G	V	S	D
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30					

oggi selezionato chiuso

Continua



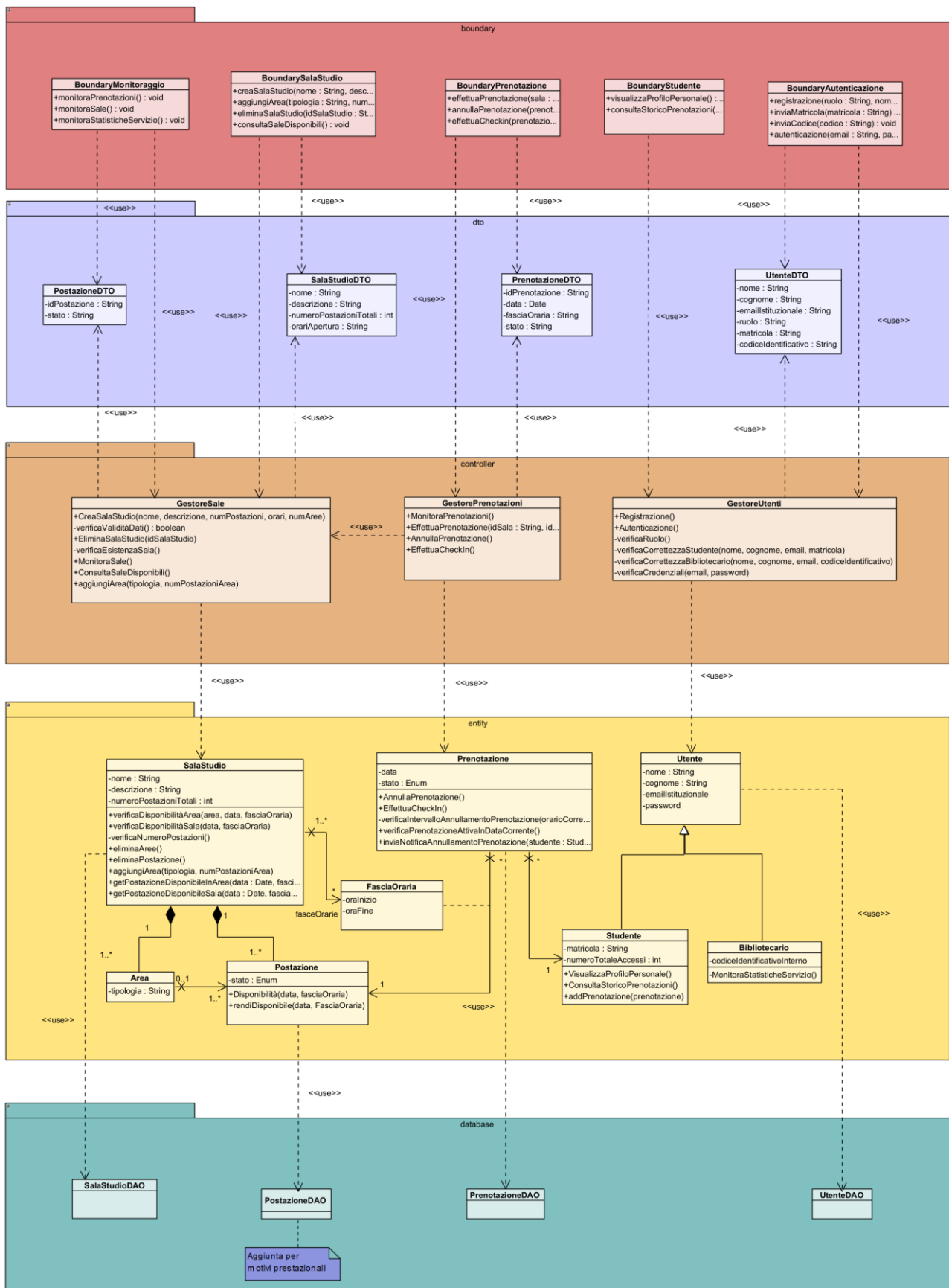
4.2 Diagramma delle classi

Riportare il diagramma delle classi di progettazione che rispetti il modello architetturale del BCED (senza violarne le regole di dipendenza fra i livelli). Il class diagram dovrà includere le classi Entity (corrispondenti a quelle di dominio), nonché le Classi Boundary, Controller, e Dati (ossia le classi DAO o Wrapper per l'accesso al Database)

Il diagramma dovrà essere organizzato utilizzando i package per raggruppare le classi dello stesso layer.

In questo diagramma saranno inoltre documentate le scelte di progetto fatte, come ad esempio:

- Reificare eventuali classi associative del diagramma delle classi di analisi.
- Specificare argomenti e tipo di ritorno delle operazioni (per quelle più significative, coinvolte nei casi d'uso sviluppati fino alla implementazione).
- Decidere i versi di navigabilità delle associazioni



4.2.1 Traduzione classi ed associazioni

1. Ereditarietà e Gestione degli Utenti

Nel passaggio dal modello di analisi a quello di progettazione, si è scelto di mantenere la gerarchia di ereditarietà per le classi del dominio relative agli attori del sistema. La classe astratta *Utente* fattorizza gli attributi e i metodi comuni (nome, cognome, email istituzionale, password e ruolo), mentre le classi concrete *Studente* e *Bibliotecario* specializzano il comportamento e gli attributi specifici (rispettivamente matricola e codice identificativo). Per quanto riguarda le associazioni, si è stabilita una navigabilità bidirezionale tra lo *Studente* e la classe *Prenotazione* (relazione 1 a molti). Questa scelta progettuale è giustificata dalla necessità dello studente di recuperare il proprio storico e le prenotazioni attive all'interno del "Profilo Personale" (che è stato integrato direttamente come responsabilità della classe *Studente* tramite i metodi `VisualizzaProfiloPersonale()` e `ConsultaStoricoPrenotazioni()`). Di conseguenza, lo *Studente* mantiene una collezione delle proprie istanze di *Prenotazione* (gestite tramite il metodo `addPrenotazione(prenotazione)`), evitando dipendenze cicliche e un accoppiamento bidirezionale superfluo.

2. Composizione strutturale (SalaStudio, Area, Postazione)

La modellazione morfologia della biblioteca è stata evidenziata una centralità data dai due contenimenti stretti di *SalaStudio*. Nel diagramma di progettazione, questa centralità è stata tradotta introducendo una relazione di composizione doppia: dalla *SalaStudio* verso la classe *Area* e, parallelamente, dalla *SalaStudio* direttamente verso la classe *Postazione*. Questa scelta garantisce che il ciclo di vita sia delle aree sia di tutte le postazioni sia strettamente subordinato a quello della sala studio in cui sono collocate.

La scomposizione logica prevede che la classe *Area* mantenga un'associazione verso le istanze di *Postazione* per tener traccia della loro allocazione interna. Tuttavia, per gestire lo scenario in cui alcune postazioni non risultino associate ad alcuna area specifica (rimanendo dislocate liberamente nella sala), il legame tra *Area* e *Postazione* è stato configurato come un'associazione semplice con molteplicità minima pari a zero dal lato dell'area.

A livello progettuale, questa struttura si traduce nelle seguenti scelte di navigabilità e attributi:

La classe *SalaStudio* detiene le collezioni principali e definitive di tutti i componenti gestiti (es. `List<Area>` e `List<Postazione>`), fungendo da punto di ingresso primario per la creazione o rimozione fisica delle risorse.

La classe *Area* mantiene una collezione di riferimenti (`List<Postazione>`) limitata esclusivamente alle sole postazioni che ricadono sotto la sua competenza logica.

Questa configurazione consente al sistema di eseguire interrogazioni flessibili: è possibile sia scansionare l'intero parco postazioni della sala (comprese quelle isolate), sia delegare all'entità *Area* il controllo della disponibilità per i soli posti ad essa associati, ottimizzando la granularità della ricerca senza introdurre accoppiamenti rigidi o ridondanze strutturali.

3. Reificazione della Prenotazione, Gestione Temporale e DTO

Il cuore logico e transazionale del sistema risiede nella gestione delle prenotazioni. Nel modello di progettazione, il concetto di prenotazione è stato **reificato** promuovendolo a una vera e propria entità (*Prenotazione*). Questa classe agisce da snodo relazionale e incapsula in sé le informazioni fondamentali come la data e il ciclo di vita della richiesta (tramite l'attributo `stato`, modellato come enumerazione).

La scelta progettuale più rilevante all'interno di questo layer riguarda la mappatura del legame tra le classi Postazione e Prenotazione. Tale legame è stato modellato ricorrendo al costrutto UML dell'**Association Class**, rappresentato dalla classe FasciaOraria. Questa soluzione traduce in modo semanticamente perfetto il vincolo del dominio: una prenotazione non occupa fisicamente una postazione in modo perenne, ma la impegna esclusivamente all'interno di uno specifico intervallo di tempo (caratterizzato dagli attributi oraInizio e oraFine).

A completamento di questa gestione temporale, il diagramma evidenzia un'associazione diretta (1 a molti) tra SalaStudio e FasciaOraria. Ciò stabilisce formalmente che le finestre temporali non sono create in modo arbitrario dallo studente, ma sono definite e governate centralmente dalla sala studio a cui appartengono, garantendo l'allineamento con i reali orari di apertura della struttura.

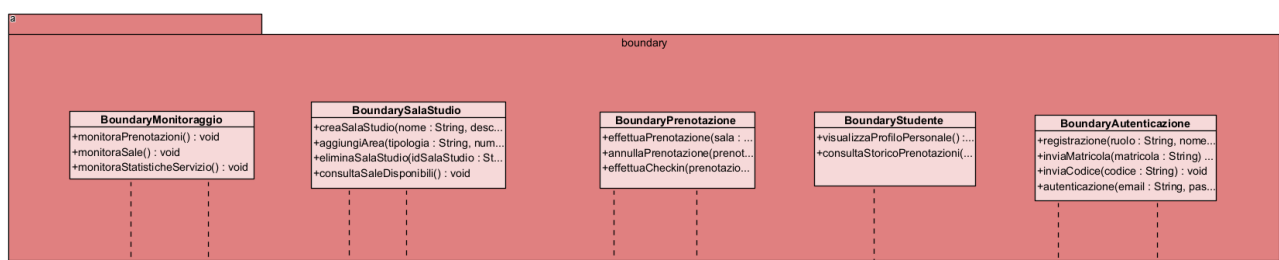
A livello di architettura generale e di passaggio dei parametri (tipi di ritorno e firme dei metodi), si è deciso di non esporre direttamente le classi del layer *Entity* ai layer superiori (*Boundary*). A tal fine, è stato implementato il pattern strutturale **DTO (Data Transfer Object)**. Classi come PrenotazioneDTO, PostazioneDTO e UtenteDTO hanno il compito di "appiattire" il grafo degli oggetti complessi. Ad esempio, all'interno di PrenotazioneDTO l'oggetto complesso FasciaOraria viene dematerializzato e tradotto in un tipo primitivo String (insieme ad altri dati come idPrenotazione), ottimizzando il payload per il trasferimento dei dati alle interfacce grafiche e preservando il principio di Information Hiding del dominio.

4.2.2 Pattern BCED

4.2.2.1 Package Boundary

Il package Boundary contiene tutti gli oggetti responsabili dell'interfaccia utente e della logica di presentazione; a questo livello tutte le classi corrispondono a delle interfacce e i relativi attributi non sono altro che gli elementi che le compongono, visualizzati a video.

Riportare il Package Boundary

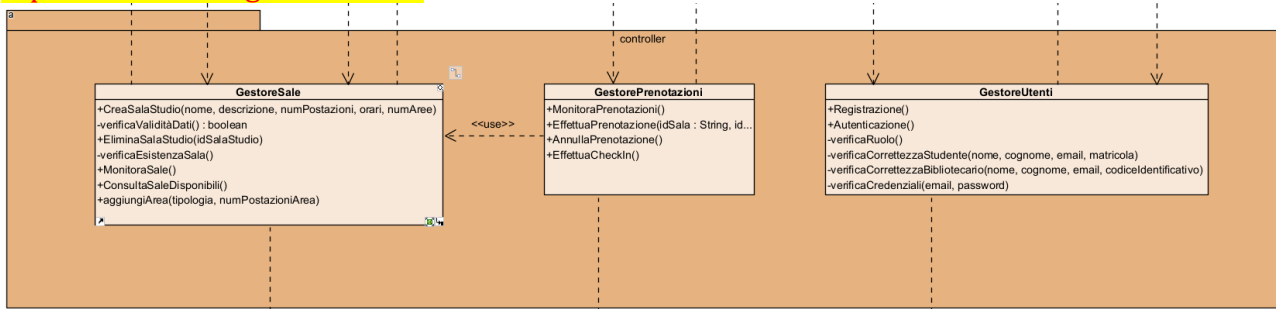


4.2.2.2 Package Controller

Questo package contiene gli oggetti che percepiscono gli eventi generati dalle interazioni con l'interfaccia utente e ne demandano la gestione all'unico componente del sistema software responsabile della gestione della Business Logic, il package Entity.



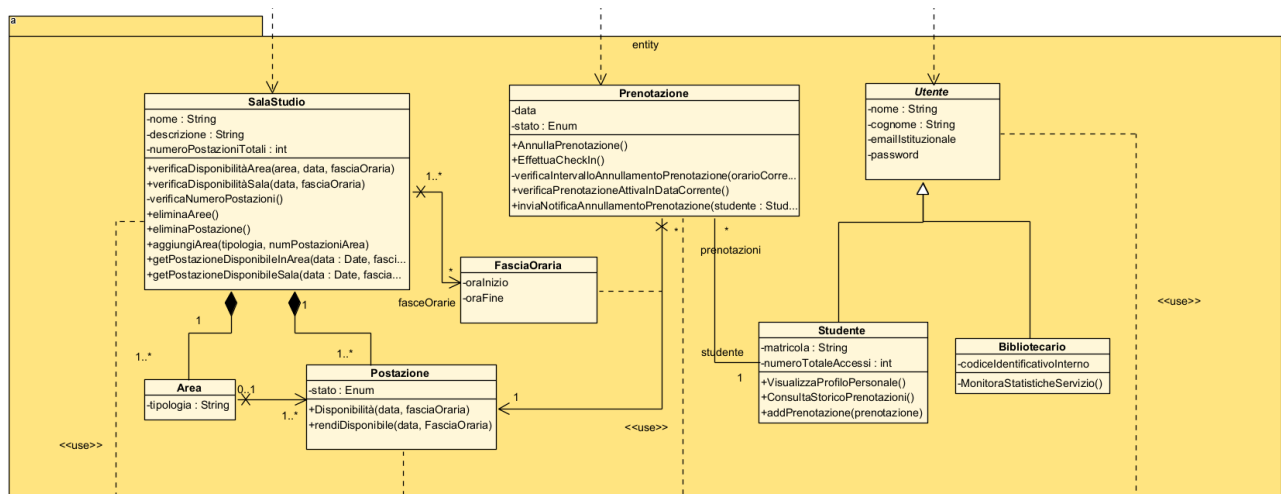
Riportare il Package Controller



4.2.2.3 Package Entity

Il Package Entity contiene tutti gli oggetti che rappresentano la semantica delle entità del dominio applicativo e corrispondono alle strutture dati presenti all'interno del database di persistenza.

Riportare il Package Entity

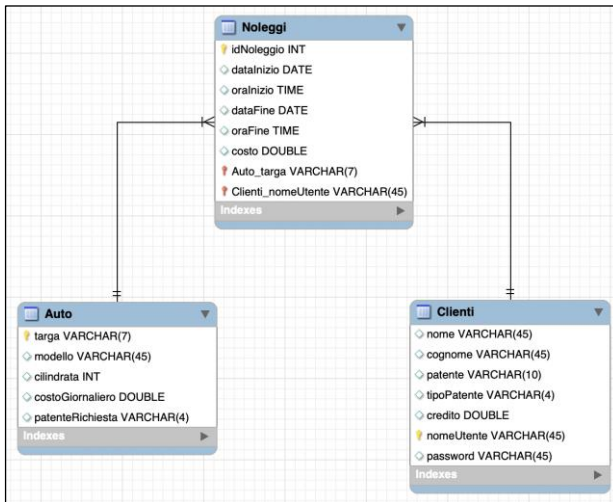


4.2.2.4 Package Database

Riportare le scelte progettuali adottate per il database di persistenza

(ossia il Modello E-R costruito ad esempio con l'editor di MySQL Workbench).

Si veda ad esempio lo schema E R allegato.



Il Package Database contiene tutte le classi responsabili dell'estrazione dei dati dal DB, esponendo una vera e propria interfaccia che di fatto rende indipendenti le classi della Business Logic (Entity) dalla tecnologia di persistenza utilizzata.

In particolare, tra le strategie di risoluzione del problema dell'**impedance mismatch**, che nasce dalla mancata corrispondenza tra il modello Object Oriented e quello relazionale, si è deciso di adottare quella delle classi **DAO** (Data Access Objects), che consiste nell'utilizzo di appositi oggetti per l'accesso ai dati.

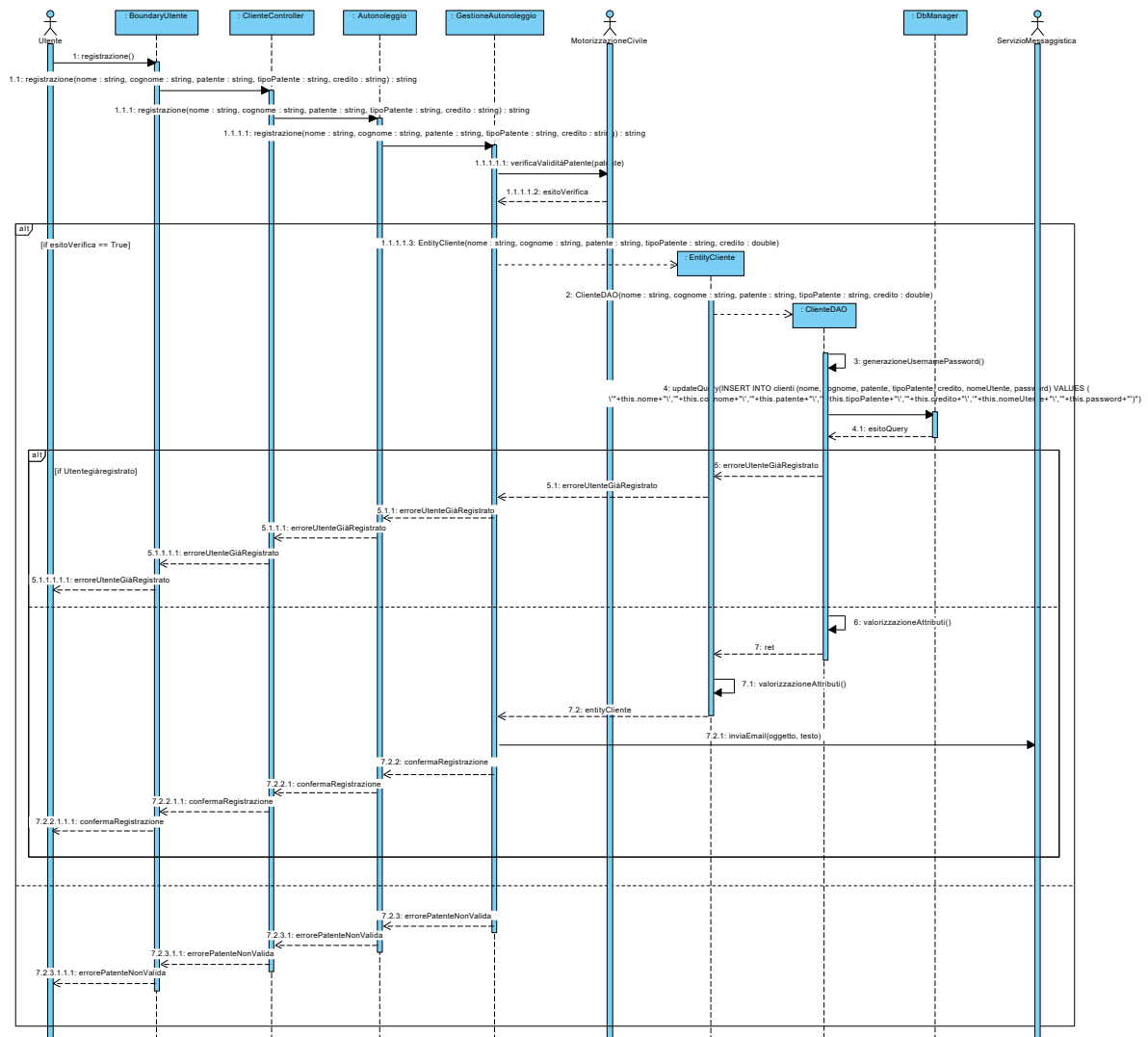
Ognuna di queste classi conterrà i metodi CRUD per l'interrogazione e la manipolazione della corrispondente classe di dominio (*query*), implementati in funzione di un'ulteriore classe, **DBManager**, che costituisce di fatto l'unico punto di accesso vero e proprio al DB, sfruttando i metodi che questa mette a disposizione.

Riportare il Package Database

4.3 Diagrammi di sequenza

Si riportino di seguito i diagrammi di sequenza di progetto per il/i casi d'uso sviluppati fino alla codifica in Java.

4.3.1 Registrazione



4.3.2 Autenticazione

4.3.3 Effettua Prenotazione

4.3.4 Crea Sala Studio

4.3.5 Elimina Sala Studio

4.3.6 Annulla Prenotazione



4.3.7 Effettua Check-in

4.3.8 VisualizzaProfiloPersonale

4.3.9 ConsultazioneSaleDisponibili

I sequence progettati sono stati fondamentali per la corretta implementazione dell'applicazione software ed ha fatto nascere la necessità di definire ulteriori classi, metodi e funzioni, che hanno arricchito passo dopo passo il [Diagramma delle Classi di Progettazione](#), fino ad ottenere la seguente versione finale:

Riportare il Diagramma delle Classi di Progettazione di Dettaglio

5. Implementazione

Non includere il codice sorgente, ma descrivere l'implementazione in Java, descrivendo gli artefatti di codifica:

- 4.1. Elencare:
 - a. package, classi, tipi di eccezione definiti
- 4.2. Elencare gli artefatti necessari per l'installazione ed esecuzione del programma, senza ovviamente l'ambiente di sviluppo come Eclipse (DB h2, eventuali librerie e versioni di Java che l'utilizzatore deve avere installati, file .class, .jar, ...)
- 4.3. Produrre un eventuale diagramma di deployment
- 4.4. Eventualmente inserire la documentazione del codice prodotta con Javadoc (relativamente alle funzionalità implementate)

5.1 Package Database

TBD

5.2 Package Entity

TBD

5.3 Package Controller

TBD

5.4 Package Boundary

TBD

5.5 Package DTO

L'introduzione di tale package, estraneo al pattern BCED, nasce dall'esigenza di mostrare sulla GUI collezioni di elementi.

Da questo punto di vista, il problema principale è proprio quello che, qualora una determinata chiamata a funzione restituisse alla GUI un elenco di entity, questa, per poterlo visualizzare correttamente a video, dovrebbe conoscere di fatto la struttura interna di tale classe Entity, ma ciò porterebbe con sé un accoppiamento troppo elevato e quindi una chiara violazione dei vincoli del pattern a livelli adottato.

Si introduce allora il concetto di **Data Transfer Object (DTO)**, un oggetto in grado di trasportare dati tra processi (nel caso in oggetto tra livelli). Le classi DTO hanno tipicamente una struttura che rispecchia quella dell'entity di cui vanno a supporto, in particolare gli attributi coincidono con quelli dell'entity che si intendono visualizzare a schermo.

5.6 Diagramma di Deployment

I diagrammi di deployment sono utilizzati per mostrare l'architettura fisica del sistema software realizzato; sono particolarmente utili per valutare, durante lo sviluppo, come un'applicazione si distribuisce tra le varie macchine.

- .

6. Testing

6.1 Test Strutturale

Costruire il Control Flow Graph per uno o due dei metodi delle classi implementate (si scelgano metodi non proprio banali), e:

- si mostri il calcolo del numero cicломatico;
 - si indichino i percorsi linearmente indipendenti;
 - si progettino i casi di test per coprire i percorsi individuati.
- Prima o a fianco del CFG riportare il codice Java del metodo.

6.1.1 Complessità ciclomatica

Si intende costruire il Control Flow Graph per due dei metodi delle classi implementate e mostrare il calcolo del numero cicломatico e i percorsi linearmente indipendenti.

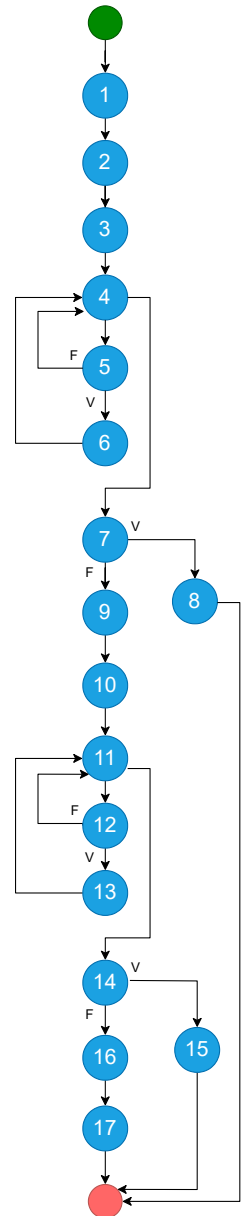
6.1.1.1 *inserisciAutoModifiche – GestioneParcoAuto*

```
public String inserisciAutoModifiche(String targa, String modello, String cilindrata, String costoGiornaliero, String patenteRichiesta) {
```

```

EntityElencoAuto elenco = new EntityElencoAuto(); (1)
ArrayList<AutoDTO> elencoAuto = elenco.getListaAuto(); (2)
boolean controlloTarga = false; (3)
for(int i=0; i<elencoAuto.size(); i++) { (4)
    if(targa.compareTo(elencoAuto.get(i).getTarga())==0) { (5)
        controlloTarga = true; (6)
    }
}
if(!controlloTarga) { (7)
    return "Errore, l'auto selezionata è inesistente!"; (8)
}
ArrayList<AutoDTO> elencoAutoDeposito = visualizzaElencoAuto(); (9)
controlloTarga = false; (10)
for(int i=0; i<elencoAutoDeposito.size(); i++) { (11)
    if(targa.compareTo(elencoAutoDeposito.get(i).getTarga())==0) { (12)
        controlloTarga = true; (13)
    }
}
if(!controlloTarga) { (14)
    return "Errore, l'auto selezionata non rientra tra quelle in deposito!"; (15)
}
EntityAuto autoDaModificare = new EntityAuto (targa); (16)
return autoDaModificare.modificaAuto(modello, cilindrata, costoGiornaliero,
patenteRichiesta); (17)
}

```



NUMERO CICLOMATICO:

1. numero di regioni chiuse del grafo + 1 = 7
2. numero di nodi predicati (4,5,7,11,12,14) + 1 = 7
3. # archi - # nodi + 2 = (22 - 17) + 2 = 7

CAMMINI:

- 1) 1-2-3-4-7-8
- 2) 1-2-3-4-5-4-7-8
- 3) 1-2-3-4-5-6-4-7-8
- 4) 1-2-3-4-7-9-10-11-14-15
- 5) 1-2-3-4-7-9-10-11-12-11-14-15
- 6) 1-2-3-4-7-9-10-11-12-13-11-14-15
- 7) 1-2-3-4-7-9-10-11-14-16-17

CASI di TEST

TC ID	Cammino Coperto	Descrizione	Condizioni Logiche	Input	Precondizioni	Output atteso	Output effettivo	Esito (PASS/FAIL)
TC_1	1-2-3-4-7-8	Percorso in cui l'auto da modificare non esiste nel sistema	La condizione al punto 7 del CFG è true	targa= CC123EE, modello=PANDA, cilindrata=800, costoGiornaliero=30, patenteRichiesta=B	La targa CC123EE non è memorizzata nel sistema	Errore, l'auto selezionata è inesistente!		PASS/FAIL
2	1-2-3-4-7-9-10-	Percorso in cui l'auto da	La condizione al punto 7	targa= CC123EE, modello=PANDA, cilindrata=800,	La targa CC123EE è memorizzata	Errore, l'auto selezionata	...	PASS/FAIL



	11-14-15	modificare esiste nel sistema, ma non esiste nel deposito	del CFG è false, La condizione al punto 14 del CFG è true	costoGiornaliero=30, patenteRichiesta=B	nel sistema, ma non è presente in deposito	non rientra tra quelle in deposito!		
..								



6.2 JUnit – Test di Unità

Riportare a scopo esemplificativo alcuni casi di utilizzo di **JUnit** per testare alcune classi del progetto, il framework di testing di unità automatizzato per il linguaggio di programmazione Java.

6.3 Test funzionale

Segue una descrizione in forma tabellare dei risultati dell'esecuzione dei test funzionali precedentemente pianificati.

REGISTRAZIONE									
TEST SUITE									
Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output attesi	Post-condizioni attese	Output ottenuti	Post-condizioni ottenute	Esito (FAIL, PASS)
1	Tutti input validi	Nome valido Cognome valido Patente valida TipoPatente valido Credito valido		{Nome: "Mario", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Cliente registrato	Si ricevono le credenziali di accesso per email	Registrazione avvenuta con successo, controlla la mail per visualizzare le credenziali di accesso	Il Cliente si è registrato e ha ottenuto l'accesso alla piattaforma	PASS
2	Nome stringa > 40 caratteri	Nome stringa > 40 caratteri [ERROR], Cognome, Patente, TipoPatente, Credito validi		{Nome: "Marioooooooooooooooooooooooooooooo oooooooooooooooooooooooooooooooooooooo ooooooooooooooooooooooooooooooooooooo", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Nome troppo lungo!		Errore, il nome inserito è troppo lungo!		PASS
3	Nome stringa con simboli	Nome stringa con simboli [ERROR], Cognome, Patente, TipoPatente, Credito validi		{Nome: "Ma-r.o", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "230.50"}	Formato Nome errato!		Errore, il formato del nome inserito non è valido!		PASS
									
12	Credito numero con simboli	Nome, Cognome, Patente, TipoPatente validi, Credito numero con simboli [ERROR]		{Nome: "Mario", Cognome: "Rossi", Patente: "NAH68I903B", TipoPatente: "B1", Credito "7@.1!"}	Formato Credito errato!		Errore, il formato del credito inserito non è valido!		PASS